

Software Design Description

Mboa Private Language Workspace

Account Isolation, Recovery and Source-grounded UML

Engineering team

Kwete Ngouba Junior Rayan

Djemtchimo Noukui Bruno Jonatan

Amina Boubakary

Version 2.0 — September 20, 2026

Design evidence for the implemented hosted business scope.

Reading guide. A4 explanations are followed by 35 full-size UML sheets. Figure links lead to readable large-format views; use PDF zoom or tiled printing. The standalone atlas contains the same diagram images.

Release boundary. Operational configuration and human acceptance remain separate from automated verification. The previous academic SDD is retained as `docs/sdd-legacy.tex`, not as deployment guidance.

Contents

1	Architecture and Design Decisions	4
1.1	Hosted application	4
1.2	Deployment topology	4
1.3	Preserved compatibility boundary	4
2	UML Conventions and Class Coverage	4
2.1	Account and web classes	5
2.2	Private storage and recovery classes	5
2.3	Existing domain groups	5
3	Identity and Request Boundary	6
3.1	Email accounts	6
3.2	Google identity	6
3.3	Browser state	6
4	Data Model and Transaction Boundaries	7
4.1	Identity and private workspace databases	7
4.2	Project and revision semantics	7
4.3	Audio publication	7
5	Language and Import Workflows	8
6	Backup and Recovery Design	8
6.1	Snapshot format	8
6.2	Preview and replacement	8
6.3	Scheduling and operations	8
7	Interface and Failure Contracts	9
8	Preserved Desktop and Academic Views	9
9	Verification, Limitations and Handover	9
10	Full-size UML Sheets	11

List of Figures

1	Hosted business components and isolated legacy storage	12
2	Same-origin deployment on one durable application host	13
3	Preserved desktop classes and optional audio ownership	14
4	Deterministic compiler classes	15
5	Language-service request classes	16
6	Translation and analysis result classes	17
7	Terminology and reference dictionary DTOs	18
8	Import and preserved academic DTO classes	19
9	Provider response classes	20
10	Provider service, configuration and grounding	21
11	Frontend class and function-component responsibilities	22
12	Hosted business user goals and supporting services	23
13	Preserved desktop collection activity	24
14	Preserved entry and editor state model	25
15	Preserved desktop audio readiness model	26
16	Preserved desktop collection sequence	27
17	Preserved desktop search sequence	28
18	Preserved desktop editing sequence	29
19	Preserved desktop deletion sequence	30
20	Preserved desktop recording sequence	31
21	Preserved desktop playback sequence	32
22	Preserved desktop statistics sequence	33
23	Legacy CSV transaction sequence	34
24	Project-bound source-labelled translation	35
25	Explicit assistant and bounded provider interaction	36
26	Consented import preview and reviewed handoff	37
27	Preserved compiler-analysis sequence	38
28	Preserved academic-evidence sequence	39
29	Read-only dictionary search and draft handoff	40
30	Hosted account and web-boundary classes	41
31	Private workspace and recovery classes	42
32	Email access and private request binding	43
33	Google sign-in and explicit account linking	44
34	Validated backup replacement and safety recovery	45
35	Guarded workspace restore activity	46

1 Architecture and Design Decisions

1.1 Hosted application

Figure 1 shows the browser, web boundary, account middleware, business routes, request-selected private storage and optional providers. FastAPI serves both the built React application and the API. This is a same-origin web application, not the previous unauthenticated shared CSV service.

1. **Identity before storage.** Every private API request resolves a verified account cookie before selecting a server-owned account directory. Client project identifiers are checked within that account.
2. **Request-local binding.** Context variables hold the identity and dataset adapter only for that request. Private data selection never mutates global CSV paths. Initialization and blocking I/O use the thread pool.
3. **Durable, bounded single-host storage.** Each account uses SQLite plus a cooperative file lock. The design does not assume a distributed database, ephemeral filesystem or multi-host lock service.
4. **Explicit persistence and processing.** Translation/chat results remain transient until saved. File previews do not approve or save terms. Provider processing is labelled and bounded; local exact lookup avoids it.
5. **Recoverable replacement.** Revision restore is version-checked. Workspace restore validates first, creates a safety backup and commits logical rows only after safe media staging.

1.2 Deployment topology

Figure 2 shows a user's browser reaching an HTTPS reverse proxy. All paths reach one FastAPI/Uvicorn application on loopback or a firewalled private interface. The operator preserves the public Host and forwarded protocol/client identity, and configures trusted proxy addresses. Arbitrary internet clients must not supply trusted forwarding headers.

The backend serves `frontend/dist`. Vite is optional during development, not the production static host. Local testing uses the one-origin launcher and loopback HTTP. Production requires a real HTTPS origin, SMTP, built assets and mandatory authentication; docs/OpenAPI are disabled.

Private storage must be persistent, outside the public web root and restricted to the service/operator account. Backups on that disk are not disaster recovery. Off-host recovery must include the identity database and configuration under separate protection, not only user-downloadable workspace ZIPs.

1.3 Preserved compatibility boundary

The desktop collector, compiler and academic export modules remain in source. Without a hosted adapter, dataset functions still use the legacy CSV/audio locations. The business factory defaults to authenticated operation and refuses academic routes in that mode. Archive fixtures explicitly disable authentication; production forbids doing so. No legacy profile, identity, CSV row or screenshot is assigned to a new account.

2 UML Conventions and Class Coverage

The inventory covers **75 authored production classes**: 73 Python class declarations, the generated lexer `Token`, and the TypeScript `ApiError`. Tests, maintenance tools, external framework types and erased TypeScript interfaces are not counted. Eleven class views distribute these classes into readable responsibility groups.

- Hollow triangles point to base classes. Dotted realization arrows point to a protocol contract; they do not claim Python nominal inheritance.
- Plain associations describe actual references or nested values and show multiplicities. Inheritance and module dependencies do not have cardinalities. Nested JSON alone does not establish exclusive lifecycle composition.
- Rectangles marked function/module/hook are not invented runtime classes. Framework types and cross-view references are marked explicitly.
- Every displayed synchronous or awaited message has an explicit receiver activation and matching dashed reply. Callers remain active while awaiting results. Actor activations represent participation, not CPU execution.
- Use-case include arrows point to mandatory reused behavior; extend arrows point toward the base use case. Activity branches use exclusive guards. State views represent logical editor/device states, not nonexistent enums.

The checker validates inventory completeness, per-view declarations, sequence activation balance, atlas mapping, rendered-image fingerprints and PDF layout logs. It is not a general UML theorem prover or substitute for design review.

2.1 Account and web classes

Figure 30 separates deployment settings, persistent identity operations, public identity data and validated request DTOs. `AuthStore` retains one settings object; identity DTOs are returned from SQLite rows rather than retained in a fictional object aggregate. Request subclasses reuse `AuthInput`'s extra-field rejection. The web middleware retains the next ASGI application, while the log filter removes sensitive query values.

2.2 Private storage and recovery classes

Figure 31 shows `WorkspaceStore`'s structural conformance to `DatasetStorage`. One selected store retains one file-lock object; its tables persist values, not live request-model instances. `HistoryInput` holds exactly one of translation or conversation content and checks that the alternative agrees with its kind. Conversations contain 1–100 messages. Project/revision/backup input classes share explicit validation rather than bypassing storage checks during recovery.

2.3 Existing domain groups

Class view	Responsibility and current scope
Figure 3	Preserved desktop App, Tooltip, Recorder and explicit storage/audio errors. Its optional recorder has lifecycle ownership.
Figure 4	Token and bounded grammar/transformation classes. The compiler remains deterministic; no provider decides parser acceptance.
Figure 5	Typed language-service requests and bounded chat messages. Academic source options are rejected in hosted mode.
Figure 6	Source-labelled translation/analysis results, vocabulary, evidence and coverage. Coverage is not accuracy.
Figure 7	Private entry DTOs, reviewed mutations, dictionary results, metadata, health and explicit collection errors.

Class view	Responsibility and current scope
Figure 8	Import preview/candidate DTOs plus preserved academic/profile/practice DTOs. The latter are not business routes.
Figure 9	Provider response parsing classes; malformed, blocked or truncated results are not successful translations.
Figure 10	Gemini service, secret-bearing configuration, grounding value and provider errors. Hosted allowance checks precede each actual outbound attempt, including retry.
Figure 11	ApiError plus correctly labelled React functions/hooks, AuthGate and the seven business areas.

3 Identity and Request Boundary

3.1 Email accounts

Figure 32 separates registration, email verification, password sign-in and a later private mutation. Passwords use Argon2id. Verification/reset tokens and session values are random and stored as hashes. Email tokens are single-use with one-hour expiry; sessions expire after seven days. Password reset revokes every existing session.

Account emails contain a link with the token in a URL fragment and require an explicit confirmation button. GET navigation alone does not consume it. Development writes an access-restricted .eml outbox. Production rejects that mode and uses SMTP STARTTLS. Delivery failures are reported, not relabelled as successful mail delivery. The outbox must never be publicly served.

Private mutations require a configured Origin and session-specific CSRF header. Identity/rate storage failures produce safe errors. Persistent anonymous, account and email/IP limits bound requests; user/global daily AI counters are charged only at actual provider attempts. Exact local lookup remains available.

3.2 Google identity

Figure 33 models the authorization-code flow with PKCE. The server stores expiring state, browser binding, verifier and nonce. The callback consumes matching state and verifies the ID-token signature, issuer, audience, expiry, nonce and verified email against Google's signing keys.

Google subject identity is distinct from an email address. An email collision with a password account requires password sign-in and explicit linking. Unconfigured Google access is not displayed as a functioning sign-in method. Automated tests use signed synthetic tokens and a mocked provider; real Google consent/domain configuration is an operator acceptance gate.

3.3 Browser state

AuthGate renders no private pages until a session is resolved. CSRF and project state remain in memory; neither tokens nor private content are stored in localStorage. Cross-tab notifications contain a random change marker only. Account changes refresh the session and clear private transient UI.

Project switching and sign-out confirm loss of drafts/recordings before remounting the application. The API helper rejects responses from an older account/project generation both before and after parsing their bodies. It does not retry a mutation under a newly signed-in identity.

Request hooks suppress duplicate in-flight mutations. After a successful settings/history mutation, read refresh explicitly cancels an older read first: otherwise a pending read could discard the

required refresh and hide a new backup. Errors after interrupted mutations state that completion may be uncertain.

4 Data Model and Transaction Boundaries

4.1 Identity and private workspace databases

Storage	Responsibility
<code>auth.sqlite3</code>	users, sessions, email tokens, OAuth state and persistent rate counters. No browser-selected user directory.
projects entries	Stable project ID, name, creation time. General is permanent. Entry ID, project reference and validated fourteen-field JSON record.
revisions	Project/entry IDs, action/time and before/after JSON. Recorded with the terminology change in the same transaction.
history	Project, kind, title, bounded content, creation and update timestamps. Saved only by explicit user action.
meta	Workspace version and owner-scoped backup catalog, schedule/state and preview metadata. Backup restore replaces logical data, not every metadata key.
audio	Validated generated filenames. An entry refers to at most one recording; several historical revisions may refer to that same immutable file.
backups	Manifest-verified ZIPs and bounded, expiring preview files.

SQLite enforces project references; parameterized queries scope records to the selected project. Per-account file locks coordinate cooperating writers across processes. They are not protection against manual operator edits or a malicious filesystem administrator. Private directories reject link/reparsed-point paths.

Normal mutations preserve IDs and provenance. Duplicate normalized text within one language is rejected. Entry evidence changes invalidate approval unless the exact new fields are explicitly reviewed. Current-project statistics do not count other users or reference-dictionary rows.

4.2 Project and revision semantics

Projects are per-account; current project selection is request-local. Project management and full-account backups deliberately use account scope, so deleting a formerly selected project does not lock the owner out of recovery. Projects containing entries, saved work or revisions cannot be deleted.

Revision restore acquires the account lock, checks the workspace version, validates required media and duplicates, then saves the recovered term with **unreviewed** status and another revision. Revisions are not a regulatory tamper-proof audit ledger: the operator controls the database and restoring an earlier workspace replaces logical revision history.

4.3 Audio publication

Recording remains in the browser until an explicit upload. Upload validation checks bounds, filename/type and content. A newly staged attachment is published with the terminology update under the storage lock; failure cleans only that new file. Existing committed recordings are retained for recovery.

There is no single database transaction covering all filesystem operations. Logical rollback and careful staging provide a narrower, explicit guarantee. A disk fault during cleanup must be

surfaced rather than claimed to undo an already committed change. Physical microphone/speaker behavior needs real-device acceptance beyond mocked failures and synthetic audio streams.

5 Language and Import Workflows

Figure 12 presents business goals, not academic roles. Visitors and owners describe session states; there is no shared-team authorization model. Optional AI suggestions extend translation only when enabled and needed. Cloud transcription extends preview only with explicit consent.

Figures 24, 25, 26 and 29 describe the domain interactions after authenticated project binding. Dataset reads in the hosted path delegate to the current private SQLite store. References are read-only resources with source provenance, not approved private entries.

Exact whole-entry evidence is used without Gemini when unambiguous. Provider requests receive only bounded selected evidence, never the full workspace, contributor identities, arbitrary notes or stored audio attachments. A consented media import is a separate provider request. Output remains a suggestion requiring human review.

Source extraction occurs on the application server. Raw import files are temporary; preview content is transient until the user explicitly saves reviewed terms/results. Browser dictation/read-aloud is separate and may contact browser-vendor services. Source permissions, sensitive-data handling and translation/transcription quality remain owner responsibilities.

6 Backup and Recovery Design

6.1 Snapshot format

A ZIP contains `workspace.json`, managed `audio/...` and `manifest.json`. The manifest maps every payload member to its SHA-256 digest. The logical export includes all projects, entries, history and revisions. Passwords, sessions, Google/Gemini credentials and server settings are excluded.

Validation rejects unsupported paths, duplicates/case collisions, linked or encrypted members, inconsistent hashes, malformed records, invalid media and missing recordings referenced by current or historical records. Archive, expanded size and count limits apply before publication or replacement. SHA-256 detects changes but does not authenticate an archive's source.

6.2 Preview and replacement

Figure 34 and activity Figure 35 make the destructive boundary explicit. Preview stores an account-scoped token, counts and current version for one hour. Restore requires literal **REPLACE**, revalidates the archive and refuses stale versions.

Before replacement the server creates a verified safety backup. New audio is staged immutably; name collisions with different content receive fresh names and all restored references are updated. Only then does one version-checked SQLite transaction replace logical rows. Failed staging/commit removes only new files. Previously committed files and the safety copy remain.

Successful replacement consumes the preview and resets the browser to General. Post-commit cleanup errors are explicit and may leave the browser uncertain: inspect saved state instead of repeating restore blindly. Reused or expired tokens cannot apply the same destructive operation again.

6.3 Scheduling and operations

Automatic backups are opt-in at 24-hour or 168-hour intervals. Maintenance checks each minute while the server runs, records attempts/errors and keeps 1–10 automatic copies. Manual and safety copies are not pruned automatically. A storage-scan failure is logged and a later cycle still runs.

Same-disk copies do not meet disaster-recovery needs. Operators must agree retention, free-space alerts, off-host consistency, access permissions, encryption, recovery-time and recovery-point objectives. Account identities require a separate complete-data-root backup. The release does not invent those objectives.

7 Interface and Failure Contracts

Surface	Design contract
Session and account APIs	<code>/api/auth</code> ; cookie identity, public session DTO and in-memory CSRF value. Registration is 202; verification/reset use explicit POSTs. Private profile/logout obey the same access checks.
Projects, history and revisions	<code>/api/workspace</code> ; project header or native-media project query. General cannot be removed. Revision restore requires <code>expected_version</code> .
Backup APIs	<code>list/create/download/delete</code> , schedule settings, multipart preview, then <code>token/version/REPLACE</code> restore. Account-scoped rather than project-scoped.
Terminology/audio	Existing validated DTOs and range-capable private media responses. No automatic adoption of global CSV files.
Safe failures	401 account required; 403 origin/CSRF; 404 absent private resource; 409 duplicate/version/capacity; 413 size; 415 unsupported type; 422 invalid input; 429 allowance; explicit storage/provider 5xx failures.
Web boundary	Same-origin assets/API, body and URL bounds, no-store private responses, no-framing/no-sniff/referrer policy and production HTTPS/HSTS. Known routes retain 405 and Allow for unsupported methods; absent API routes remain 404 rather than falling into static-file handling.

8 Preserved Desktop and Academic Views

The original source is maintained, not exposed as hosted business functionality. Its diagrams remain in the inventory so every production class and retained workflow has an explicit scope:

- Figure 13 and states 14, 15: desktop entry/editor and device lifecycle, not a hosted account state machine.
- Figures 16, 17, 18, 19: standalone collector operations against the legacy CSV.
- Figures 20, 21, 22, 23: retained desktop device, measurement and CSV transaction boundaries.
- Figures 27, 28: preserved compiler/research maintenance. These are not seven-area business UI features or evidence of genuine research completion.

The archived SDD preserves the earlier prose. Historical student details and constructed examples are not silently reintroduced into private onboarding.

9 Verification, Limitations and Handover

Automated checks combine real SQLite/cookie/account behavior, mocked external services, synthetic recordings, delayed-request regressions and real browser downloads/restoration. In partic-

ular they cover account/project isolation, password-reset revocation, CSRF, malformed Google identity, backup tampering, stale versions, missing historical audio, rollback and maintenance retry.

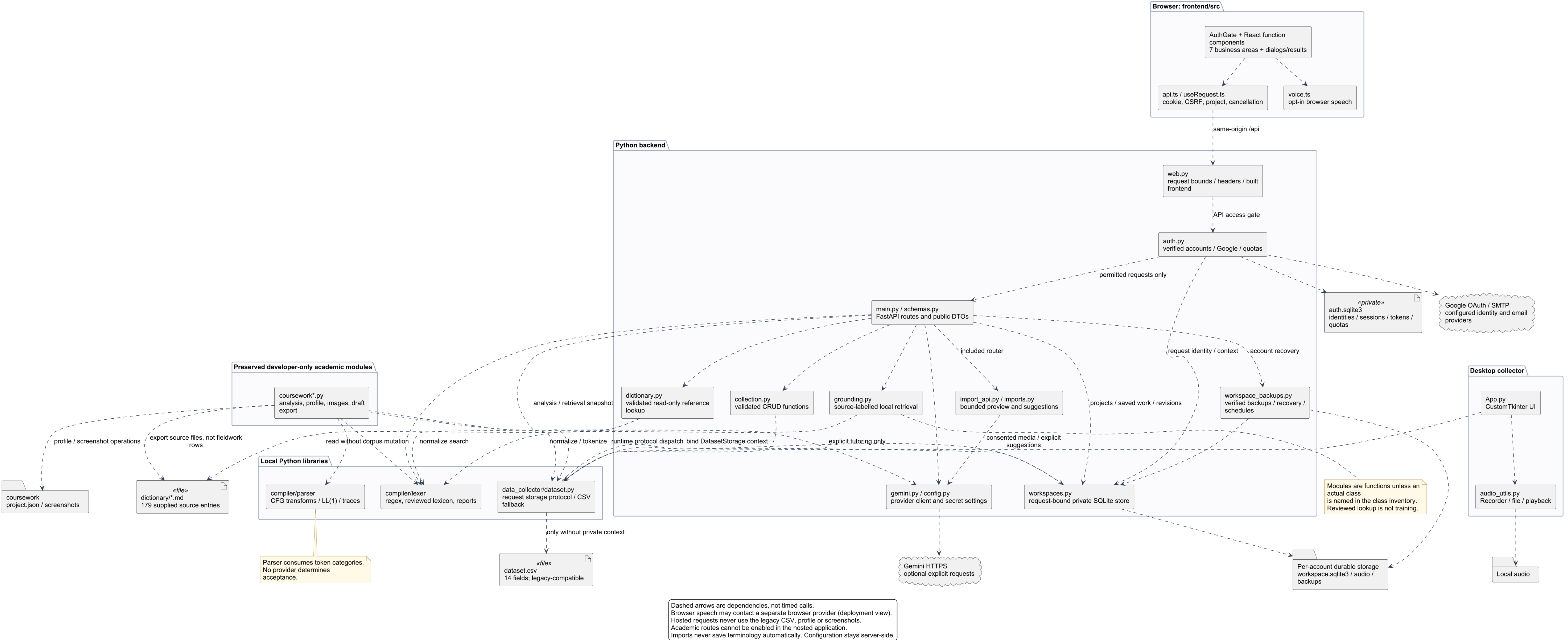
The full-stack suite uses a new, isolated temporary data directory, local development email and disabled provider configuration. It does not contact Google/Gemini or use physical hardware. Tests do not populate real business content. The verification register distinguishes current observations from previous-version counts and unexecuted remote CI.

Before launch, the owner supplies infrastructure and real mail/OAuth/provider configuration, approves content rights and retention/terms/support, evaluates language quality and physical accessibility, agrees performance/recovery targets and authorizes publication. No synthetic benchmark certifies an 8 GB reference machine or an unspecified hosted traffic level.

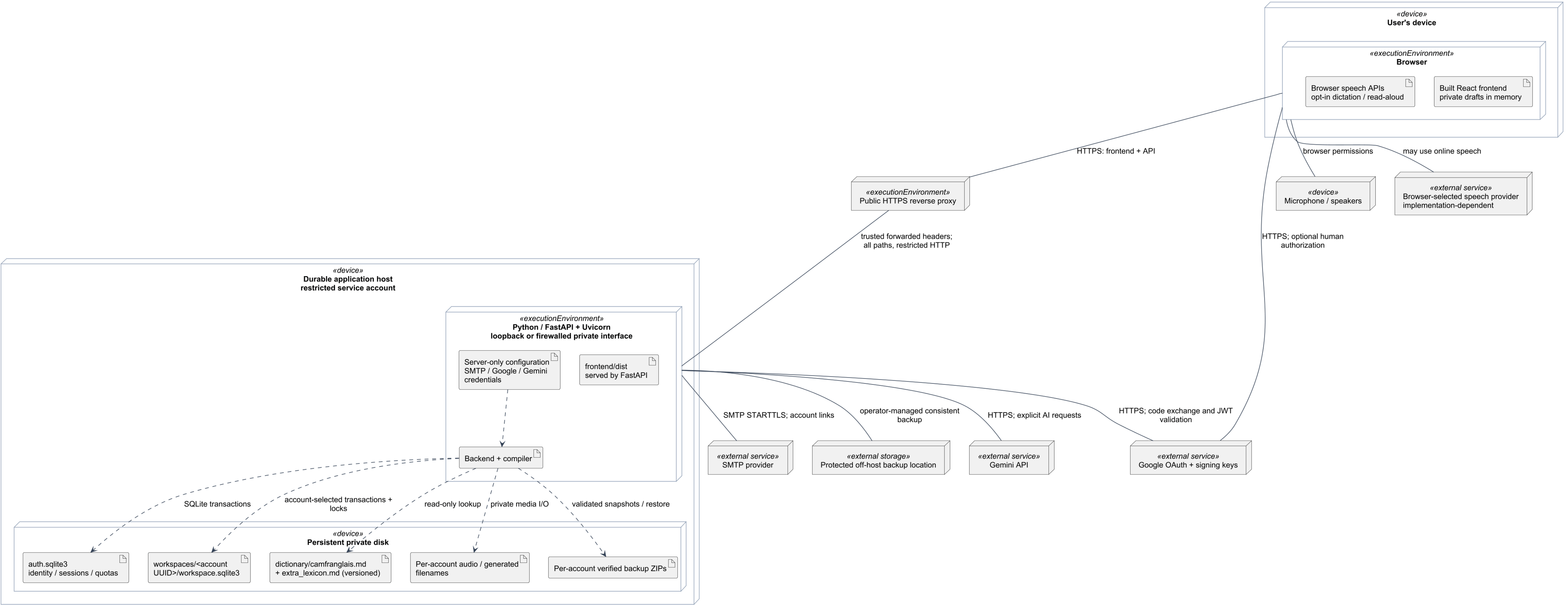
10 Full-size UML Sheets

The following pages embed the exact atlas images. New hosted views are appended to preserve established atlas page references. All displayed sequence communications have explicit activation boxes.

Hosted business components and isolated legacy storage

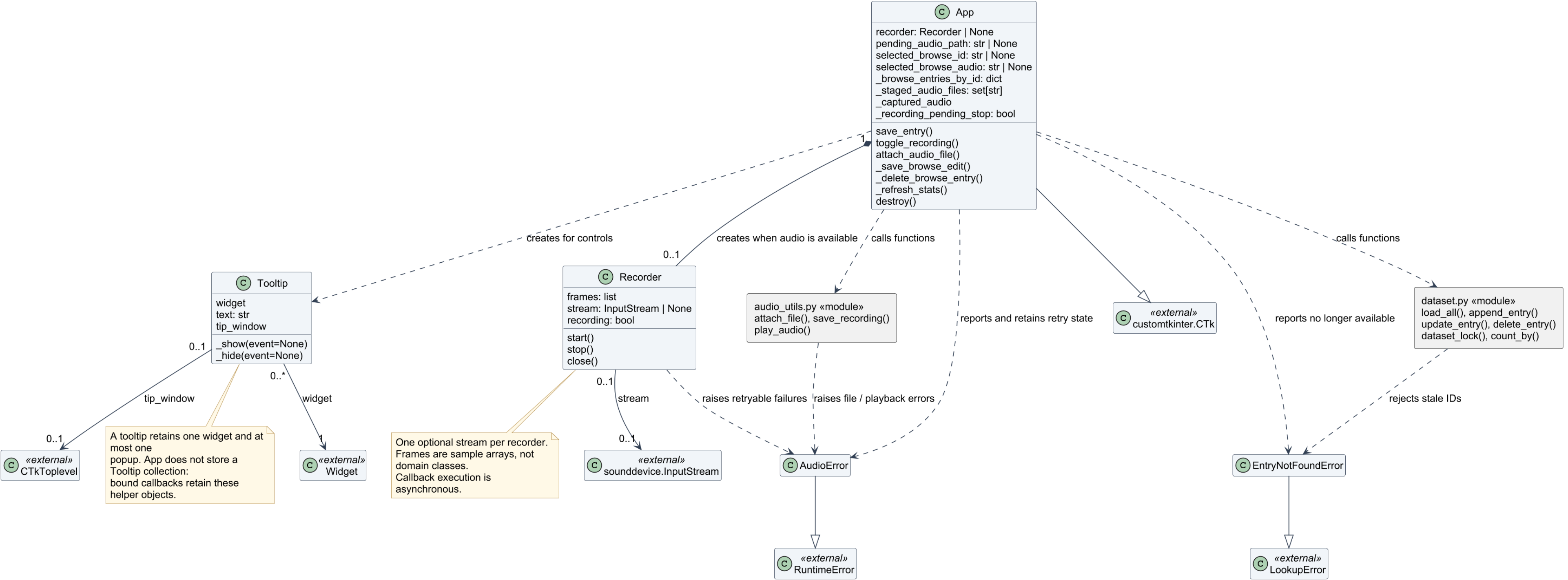


Hosted same-origin application: one durable application host



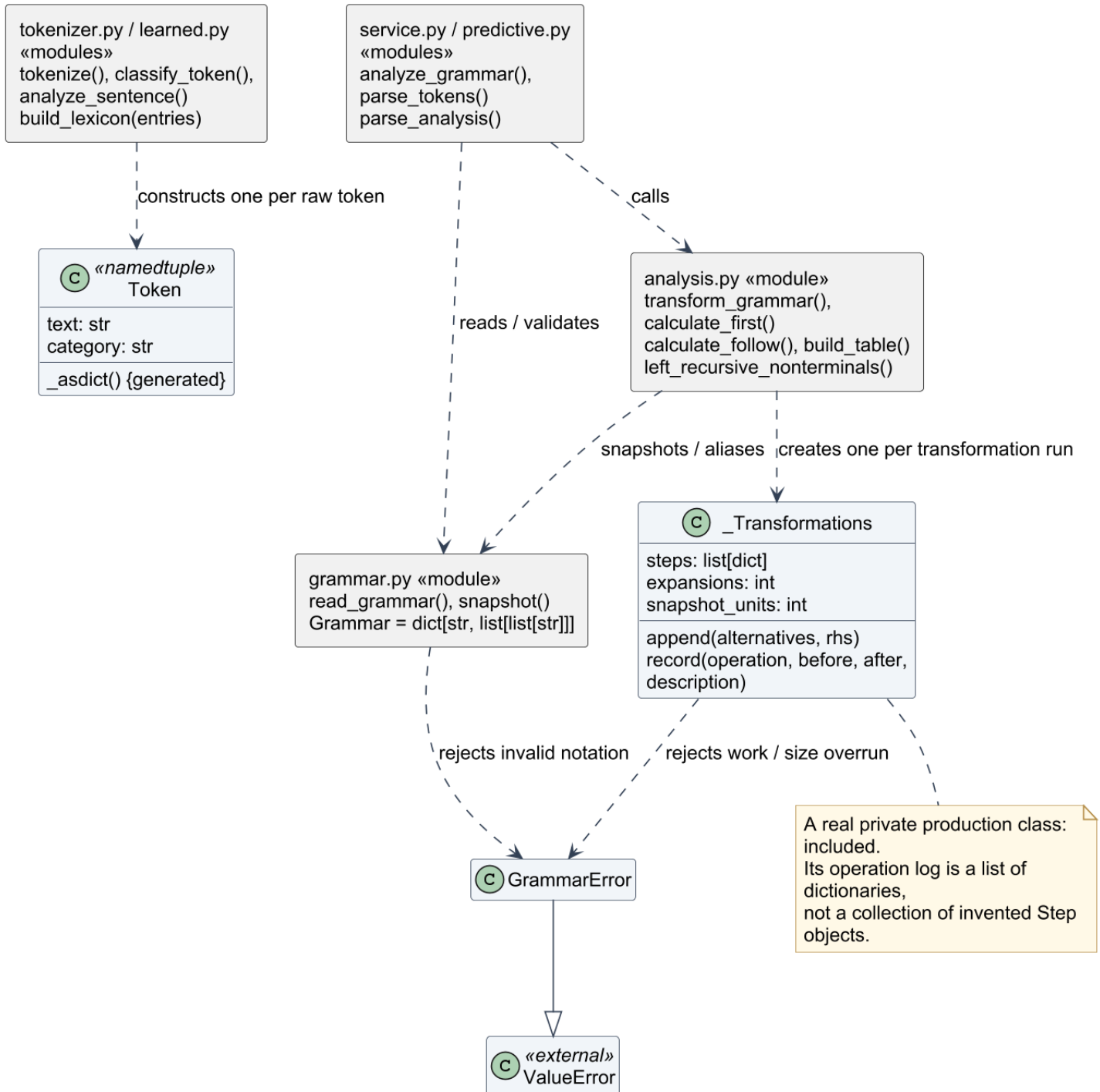
Authentication is mandatory for private APIs. CORS is not access control.
Local pre-hosting testing: browser -> loopback FastAPI, no public proxy required.
Vite is an optional development server, not the production static host.
Legacy desktop CSV/audio/profile are separate and never adopted by a hosted account.
Disk backup requires identity data too; workspace ZIPs alone cannot recover accounts.
Single durable host only: no ephemeral disk or implied multi-host database cluster.

Desktop application classes and audio ownership



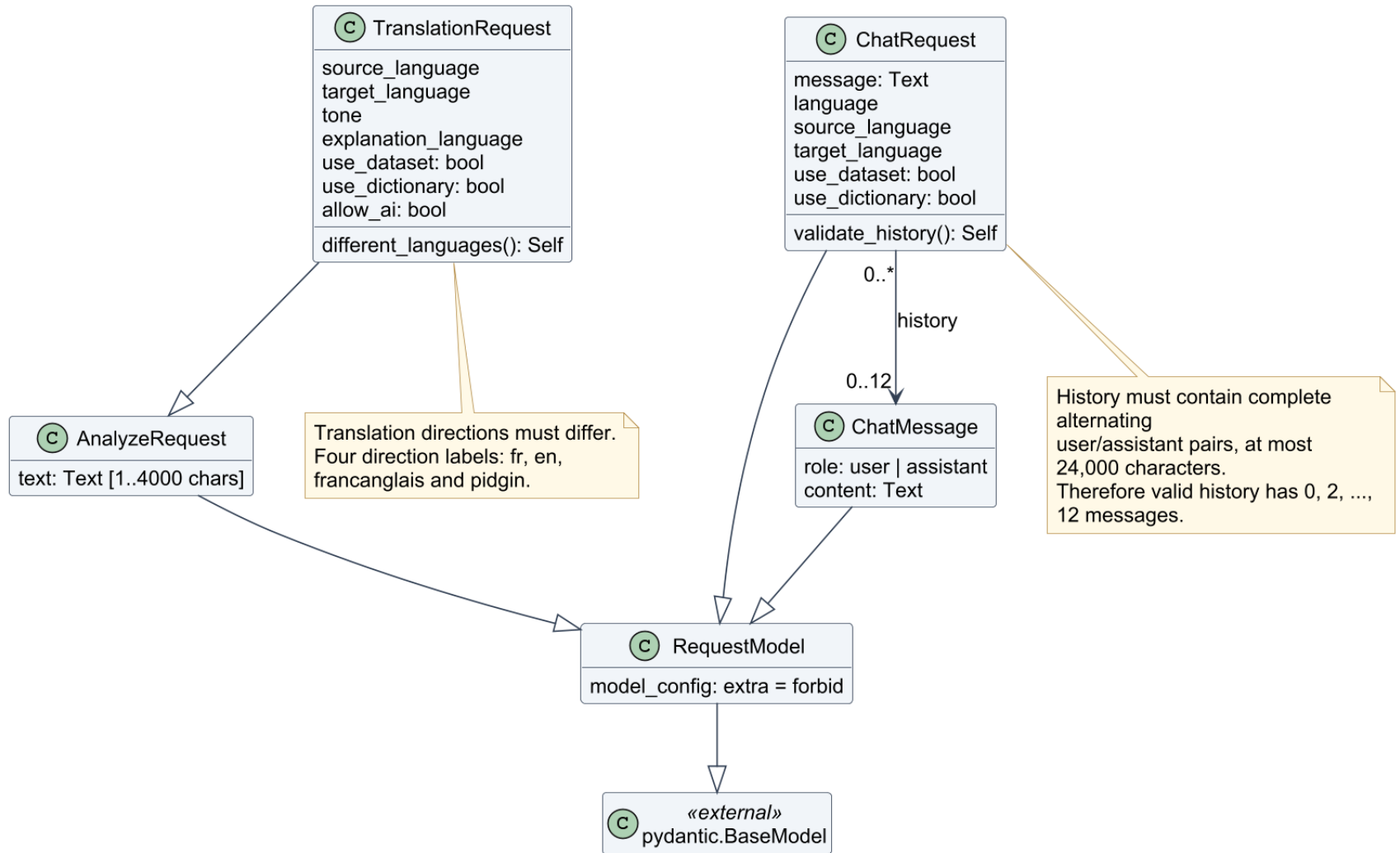
Five production classes; library types are explicitly external.
Per-instance target multiplicities express stored references.
Audio filenames are not object ownership or database foreign keys.

Compiler runtime classes and function modules



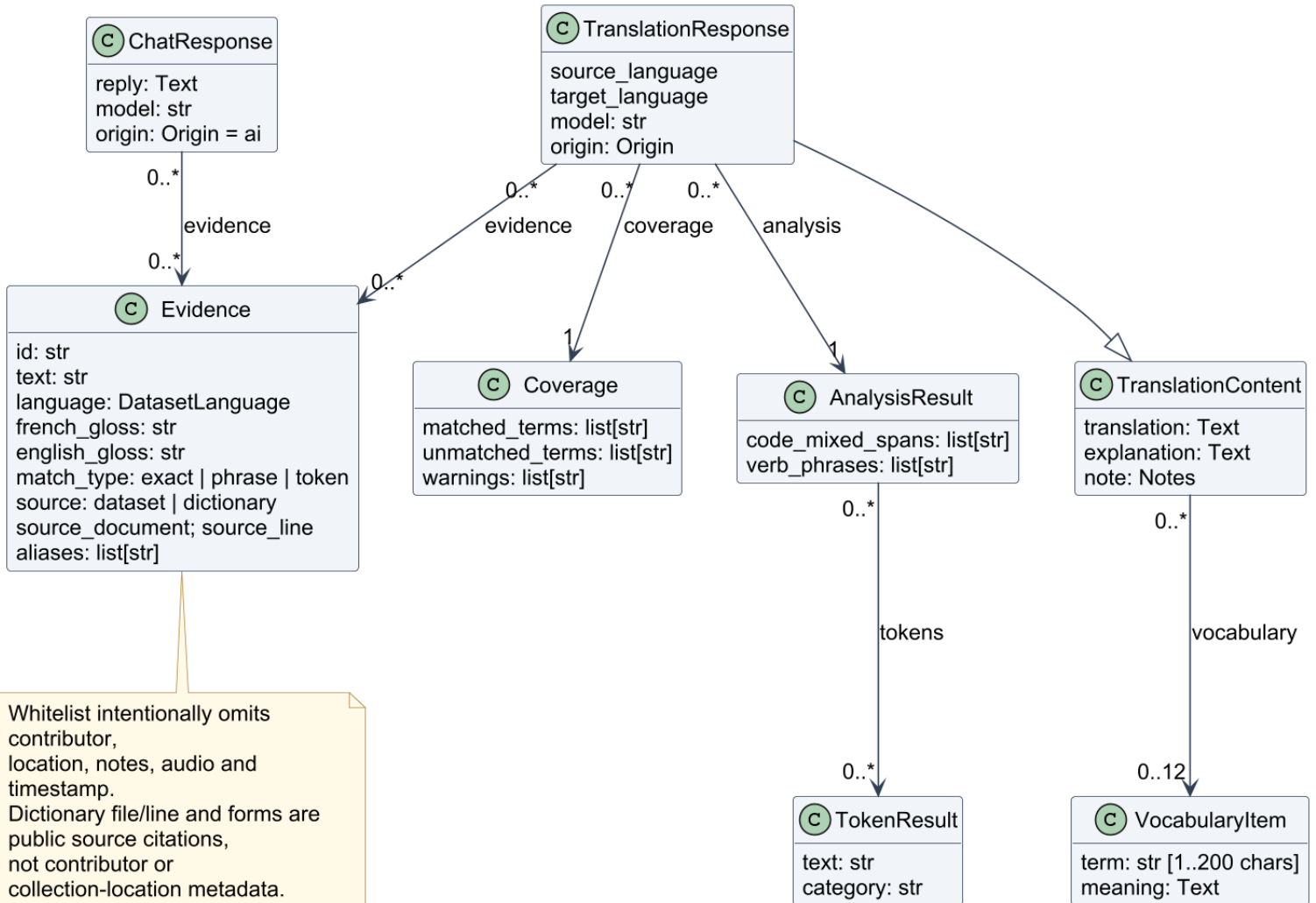
Token is an application-defined runtime class generated by namedtuple. Grammar, Production, analysis results and parse traces are aliases/dictionaries. There is no `LexerService`, `Grammar` class, `Parser` class or compiler database. Lifetimes of function-local values are not drawn as class compositions.

API request inheritance and conversation validation



All five primary classes live in backend.schemas.
Text is a constrained string alias, not an additional class.
Entry, import and coursework request subclasses are in detail views.

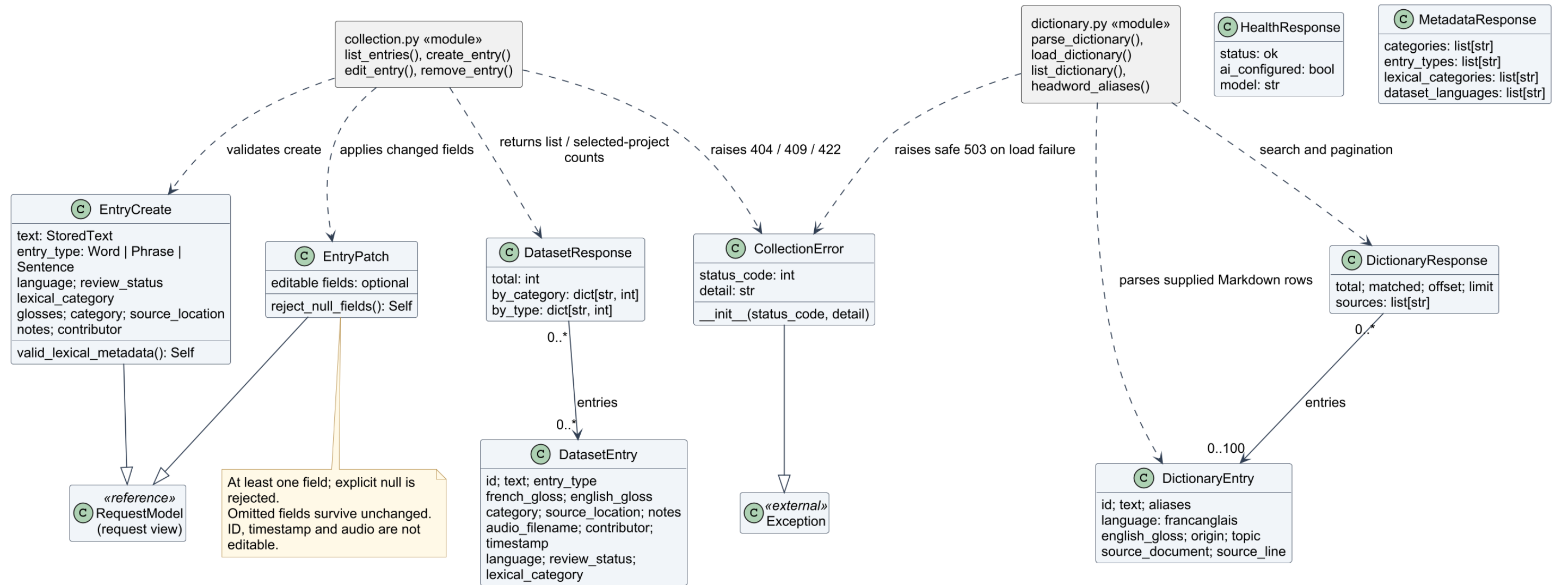
Translation and analysis result DTOs



Whitelist intentionally omits contributor, location, notes, audio and timestamp. Dictionary file/line and forms are public source citations, not contributor or collection-location metadata. Retrieval emits at most 8 evidence records, but these DTO list types alone do not enforce 8.

All eight primary classes live in backend.schemas. Each directly extends pydantic.BaseModel except TranslationResponse. Associations describe nested values, not exclusive lifetimes; Pydantic values may be shared/reused by Python callers. Validation/serialization use inherited model_validate/model_dump.

Separate collection and reference dictionary DTOs

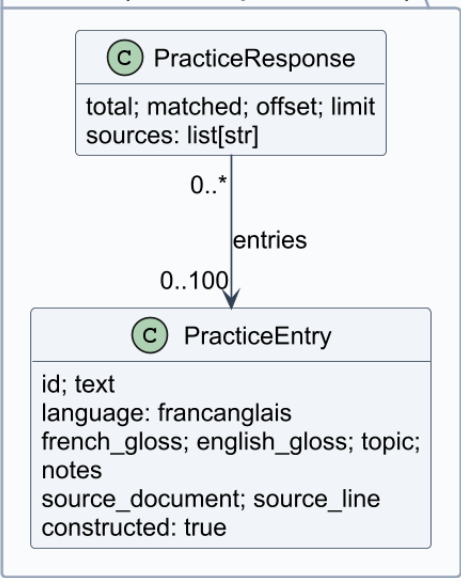


The six response DTOs directly extend pydantic.BaseModel. EntryCreate/EntryPatch inherit extra=forbid via RequestModel. DatasetEntry is an API DTO, not the storage representation: Hosted storage serializes dictionaries to private SQLite JSON rows. Desktop fallback helpers still persist dictionaries as CSV. Reference entries are read-only, not approved corpus entries. Their etymological origin does not supply a French gloss or POS.

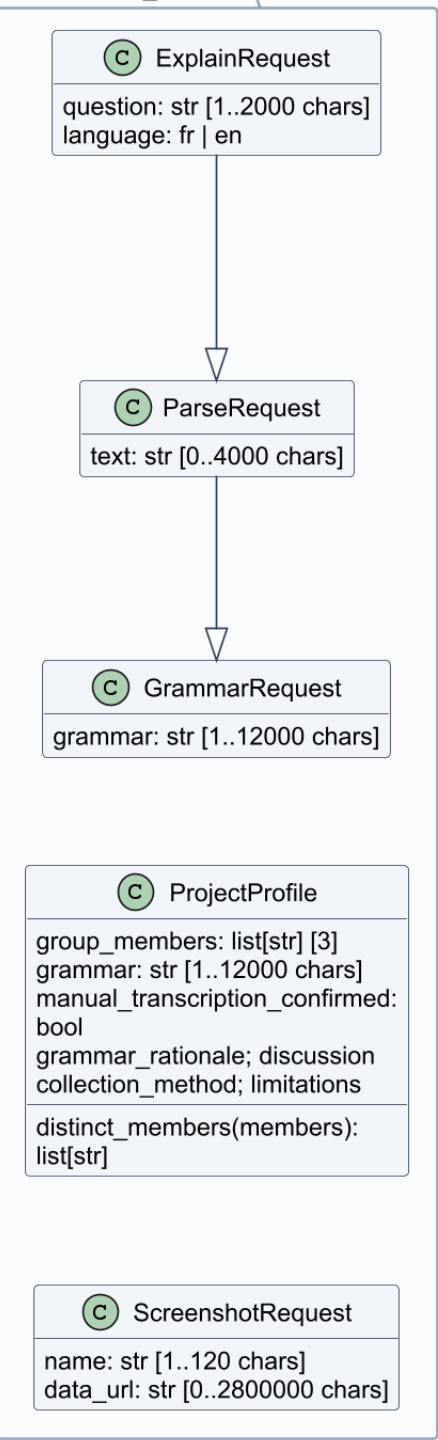
Import models and preserved academic request/result classes

backend

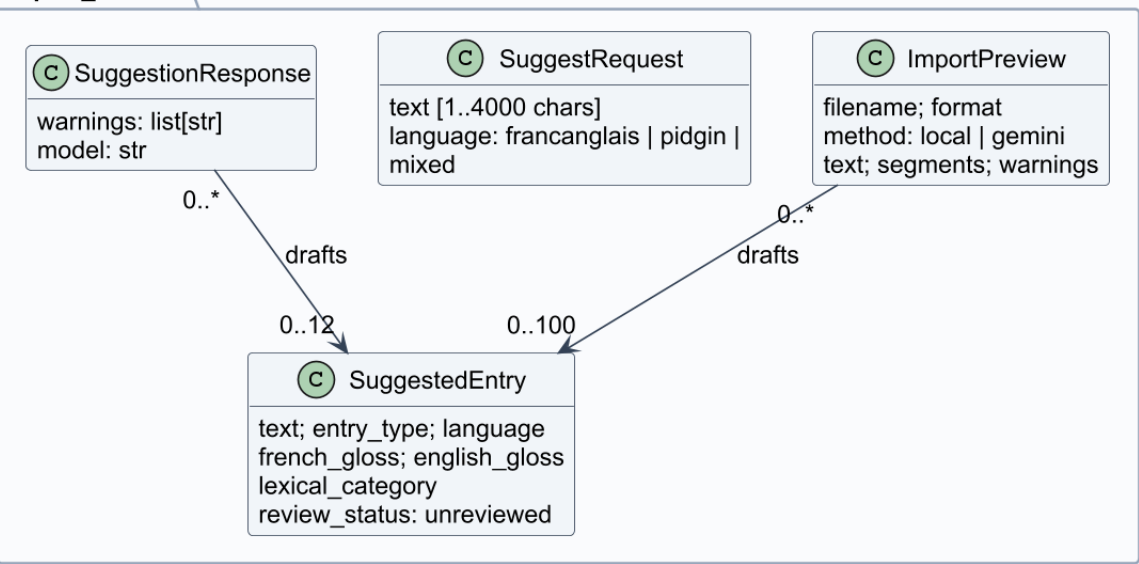
schemas (archived practice route)



coursework_models

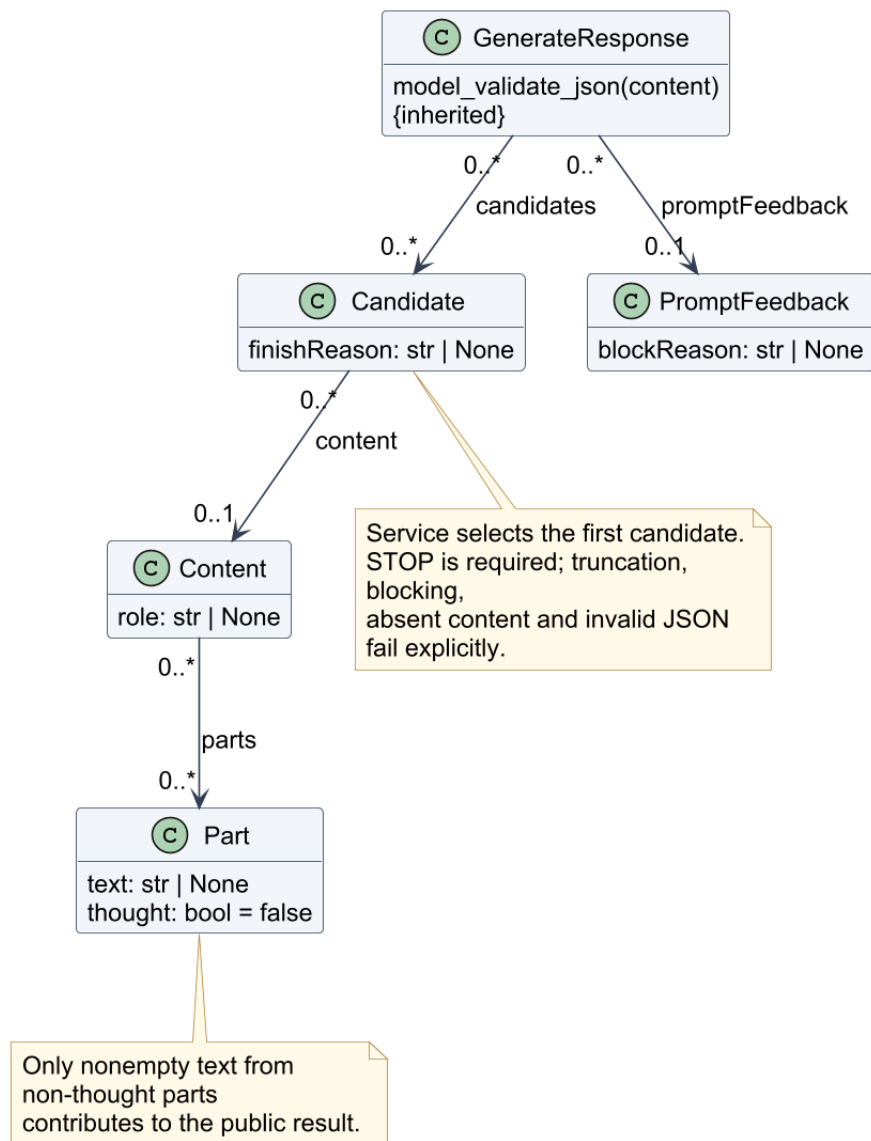


import_models



SuggestRequest and all coursework classes inherit RequestModel (directly, except ParseRequest and ExplainRequest as drawn). Other import classes directly inherit BaseModel. PracticeEntry and PracticeResponse also inherit BaseModel. SuggestedEntry additionally forbids extra fields; approval cannot be imported. Three member strings may initially be blank; nonblank names must be distinct. UI type aliases/interfaces are not additional runtime model classes. Academic DTOs remain in source but are not exposed by the hosted application.

Gemini wire-response validation classes

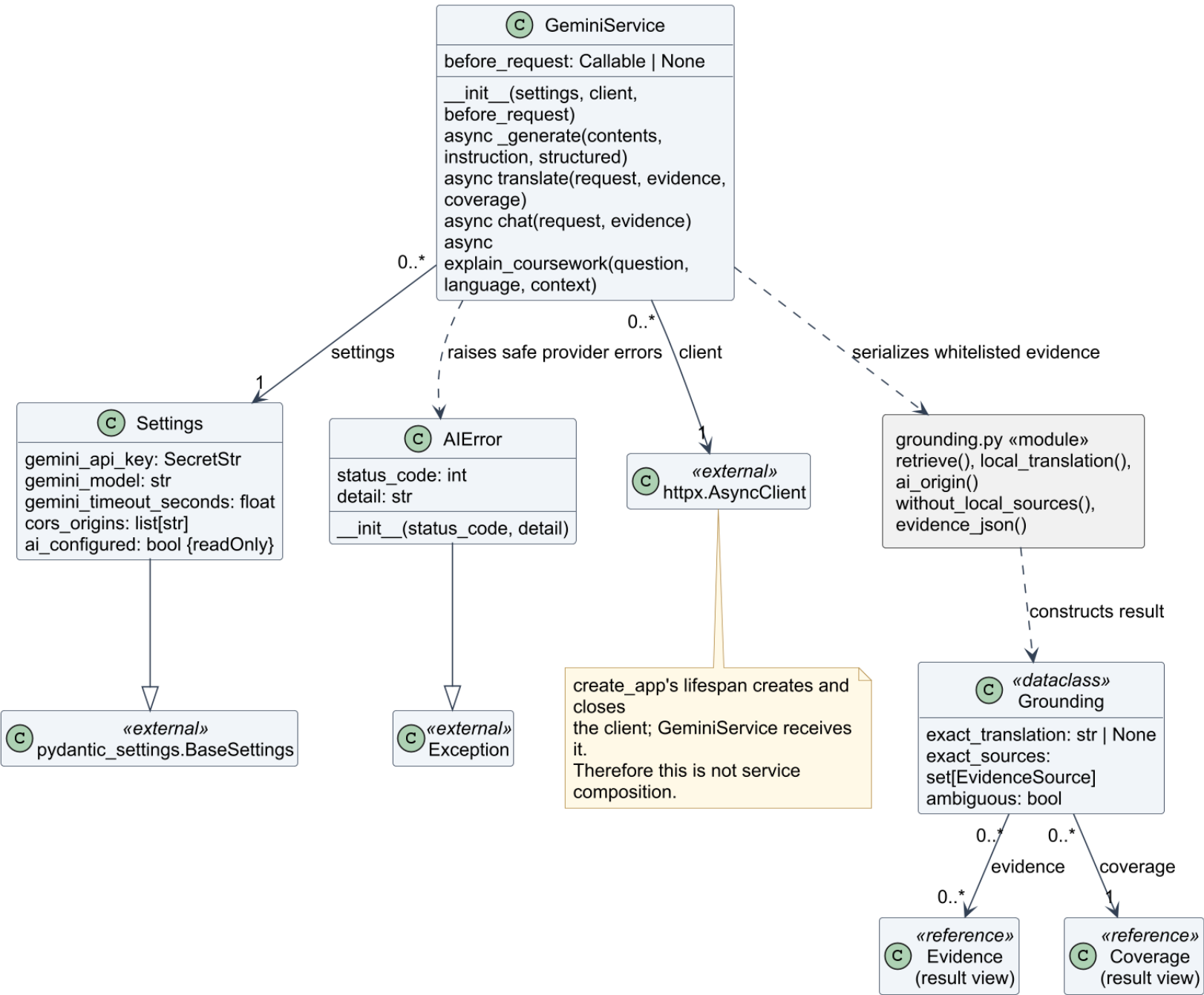


All five primary classes live in `backend.gemini` and extend `BaseModel`. Optional fields/list defaults justify 0..1 and 0..*.

They validate upstream wire data, not stored corpus entities.

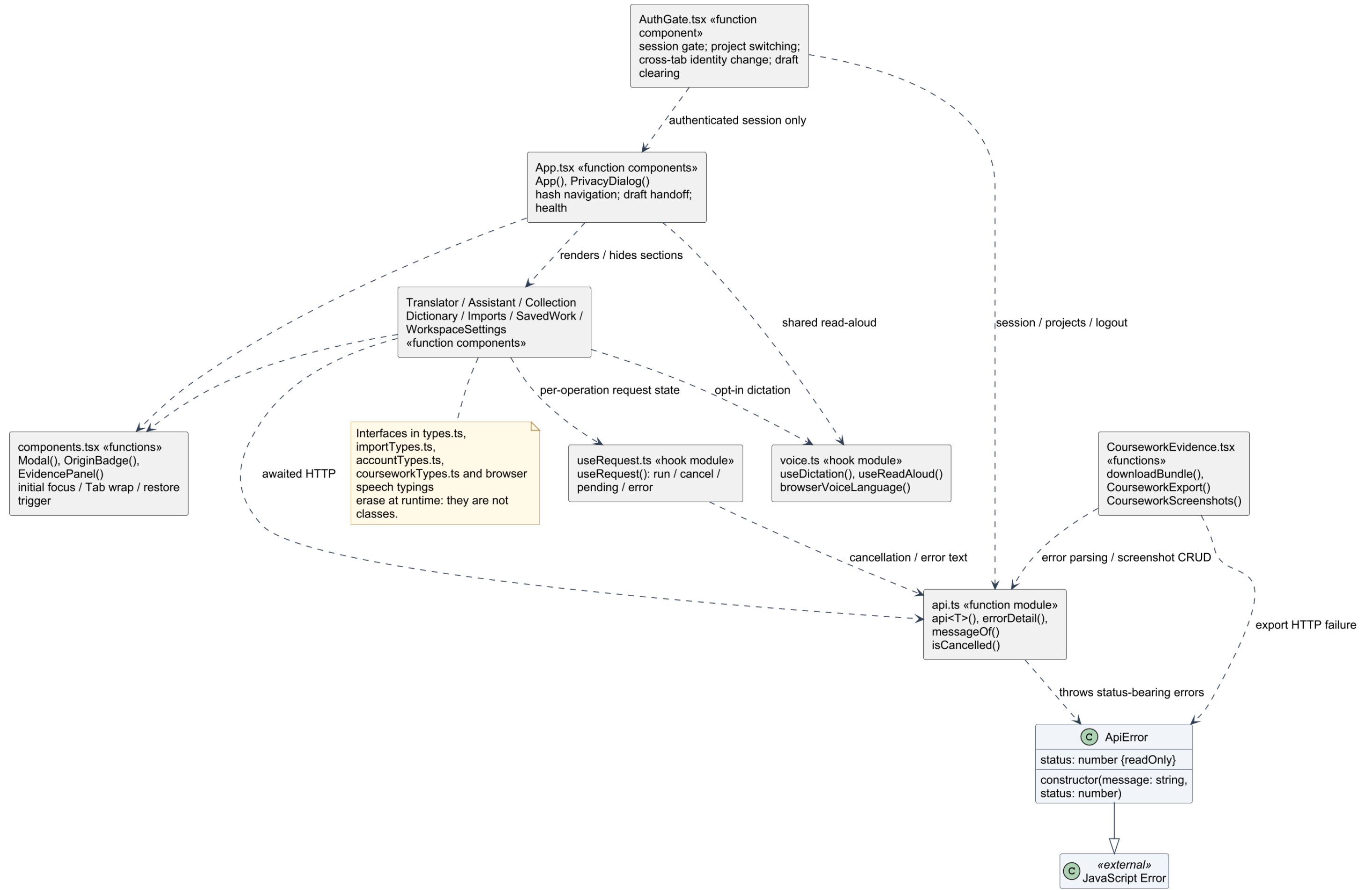
These classes are distinct from public `TranslationContent/ChatResponse` DTOs.

Actual provider service and local grounding result



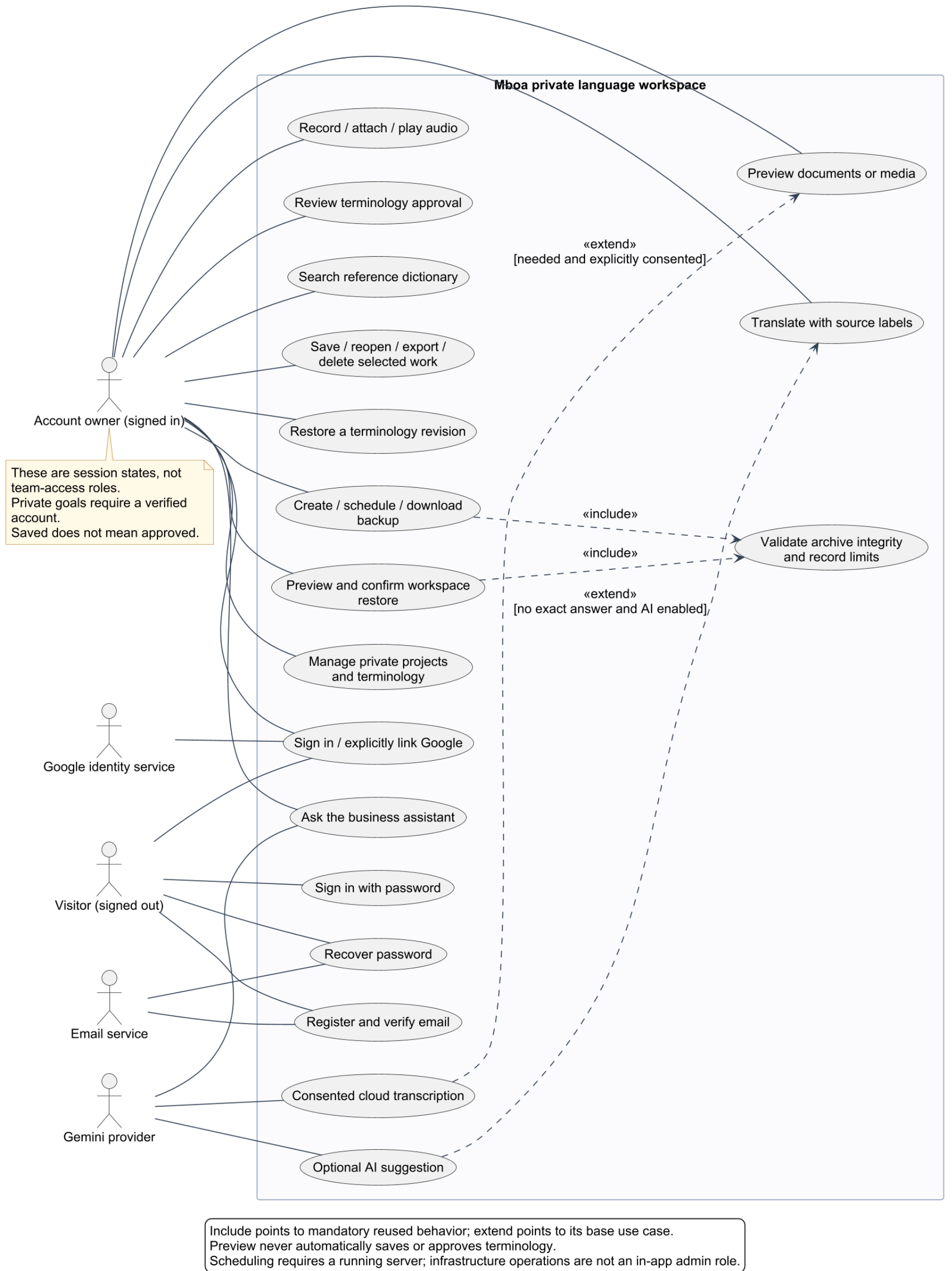
GeminiService is a real class. Retrieval and route handlers are functions. Grounding is a value holder, not a hidden training or translation service. At most 8 selected evidence records are emitted by retrieve(). Hosted quota accounting runs before every actual provider attempt, including retry.

Frontend class and function-component structure

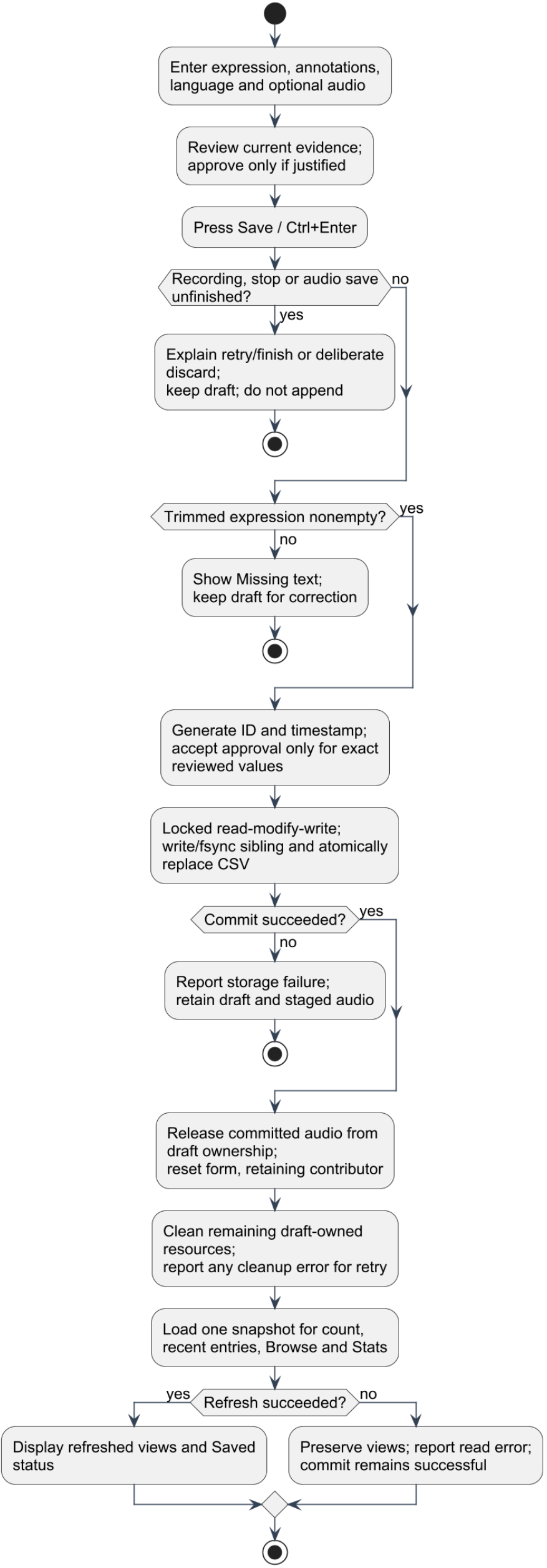


ApiError is the only authored production TypeScript class.
React function components and hooks must not be recast as service classes.
Module dependencies have no association multiplicities.
CourseworkEvidence is preserved archive code, not a business navigation destination.
api.ts attaches CSRF/project context and rejects responses from older account generations.

Hosted business user goals and supporting services

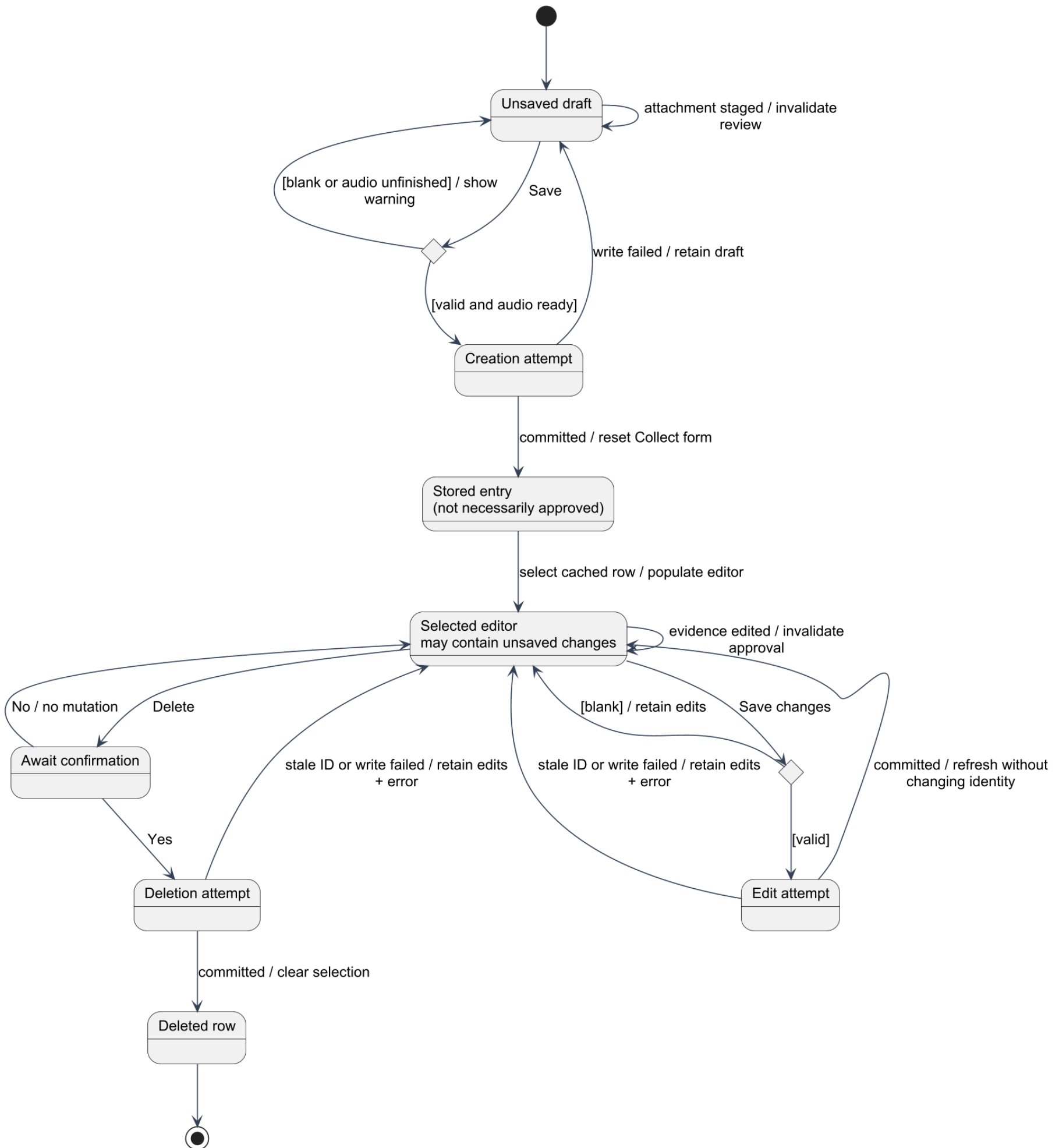


One desktop save attempt, including failure boundaries



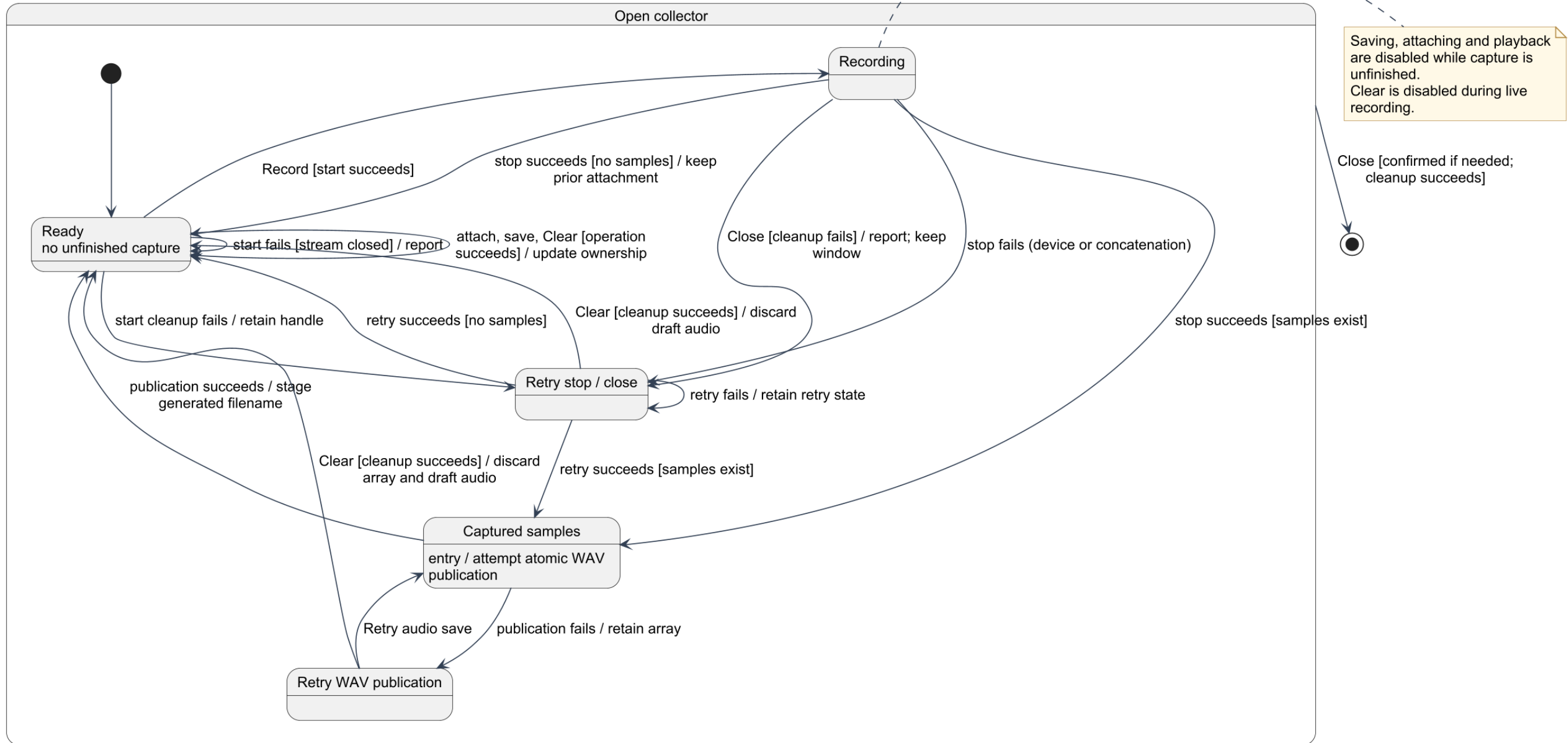
Diamonds represent exclusive guarded decisions, not parallel work. One final node ends this attempt, not the collector application. Duplicate warnings are advisory on desktop; API duplicates return 409. File staging is separate; recording details appear in the sequence view.

Logical entry and selected-editor lifecycle



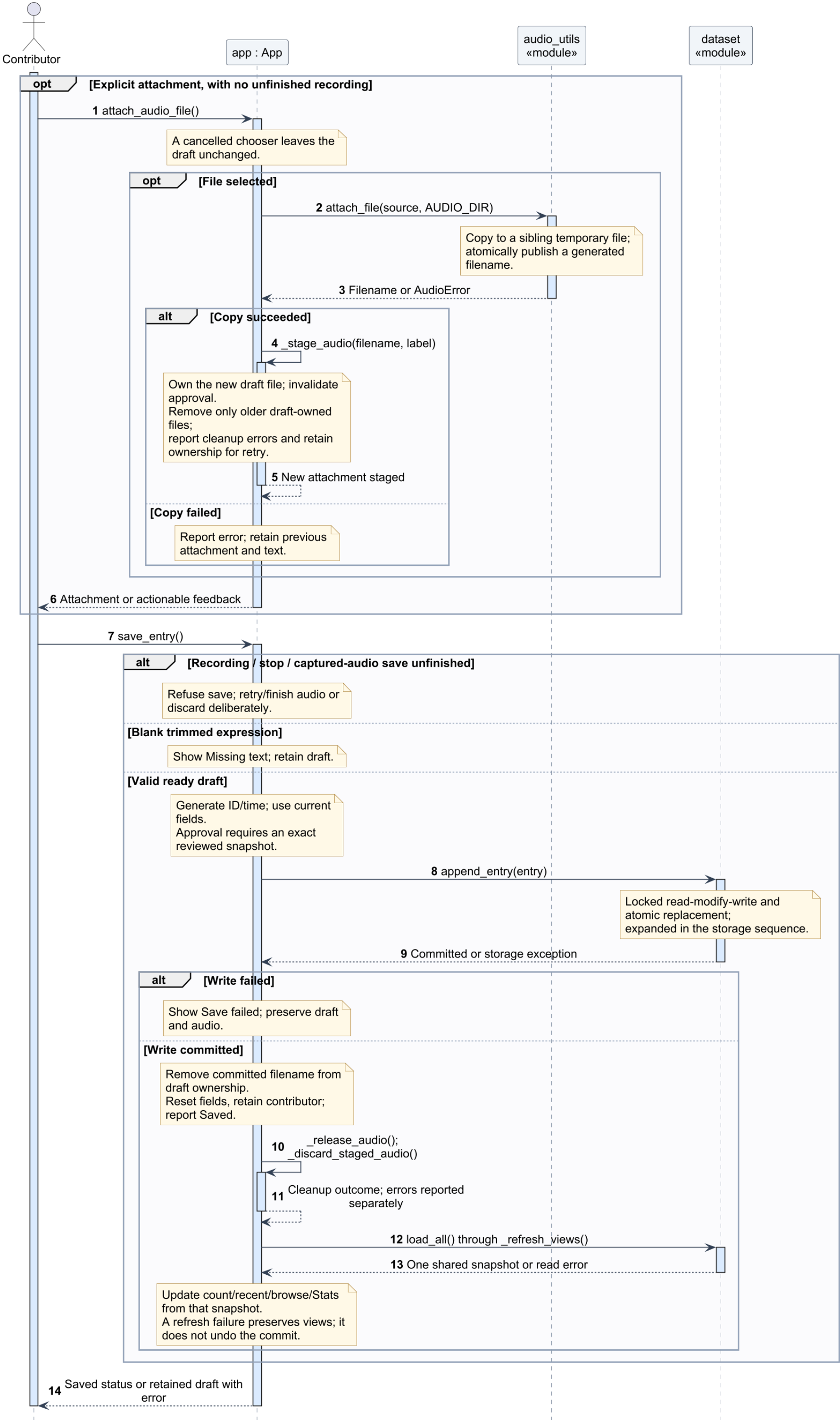
This is a logical row/editor model, not an implemented enum or whole-app lifecycle. The selected editor may outlive a row removed by another writer; stale errors are explicit. A successful commit is not reversed by later display/cleanup failure. Audio staging has a separate lifetime; deleting a saved row retains its recording.

Desktop capture readiness and retry states

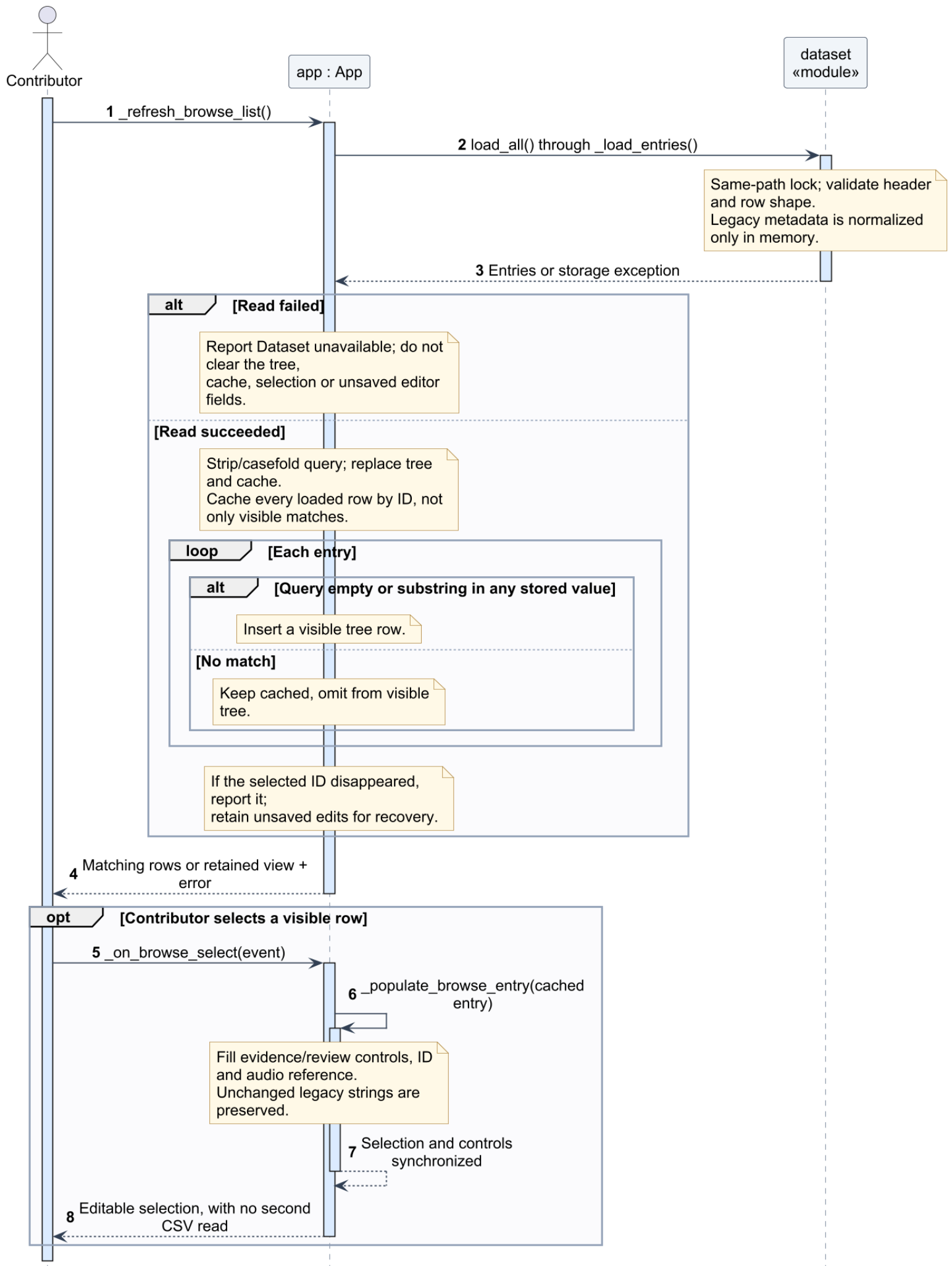


Conceptual states derive from recording, stream, pending-stop and captured-array fields. Ready may contain a staged filename; it does not mean an empty form. Failed cleanup keeps the window/retry state available, not a false closed status. Draft-owned files may be discarded; committed recordings are never owned by the draft.

Desktop attachment and truthful entry saving

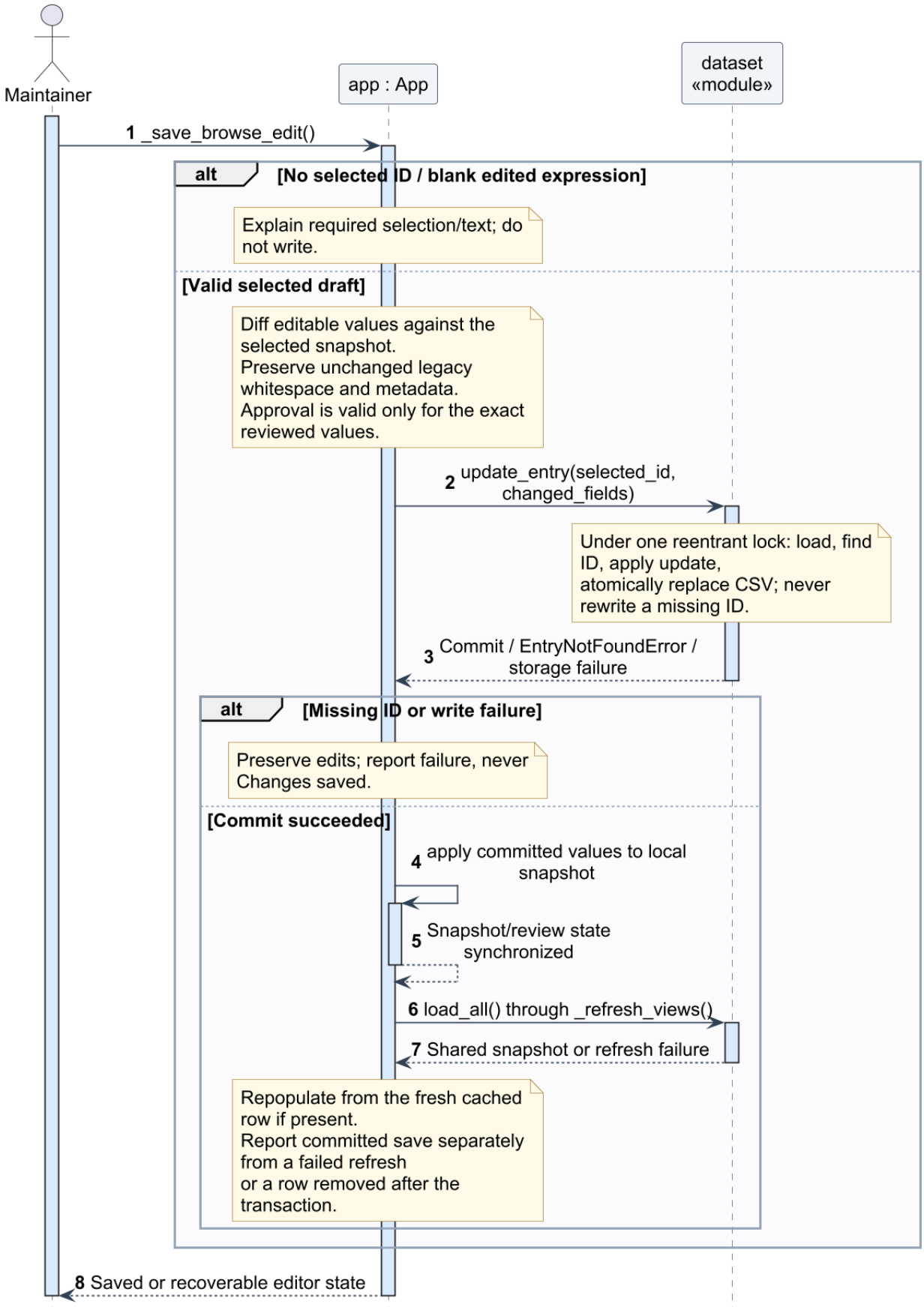


Desktop search, preserved views and cached selection



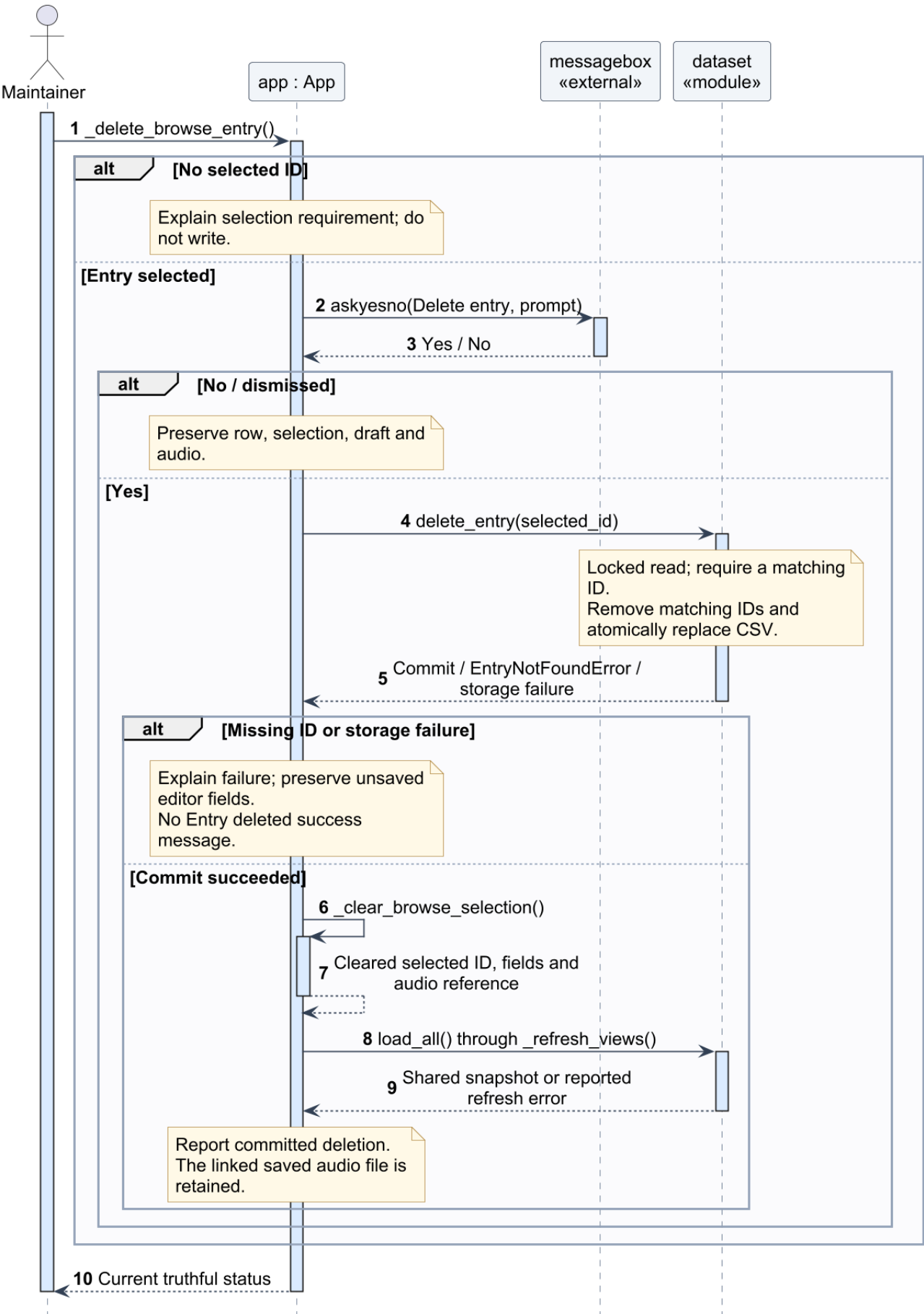
IDs should remain unique; manually corrupted identifiers are not repaired here. Filtering is read-only. Empty results are distinct from a failed read.

Desktop changed-field editing and review integrity



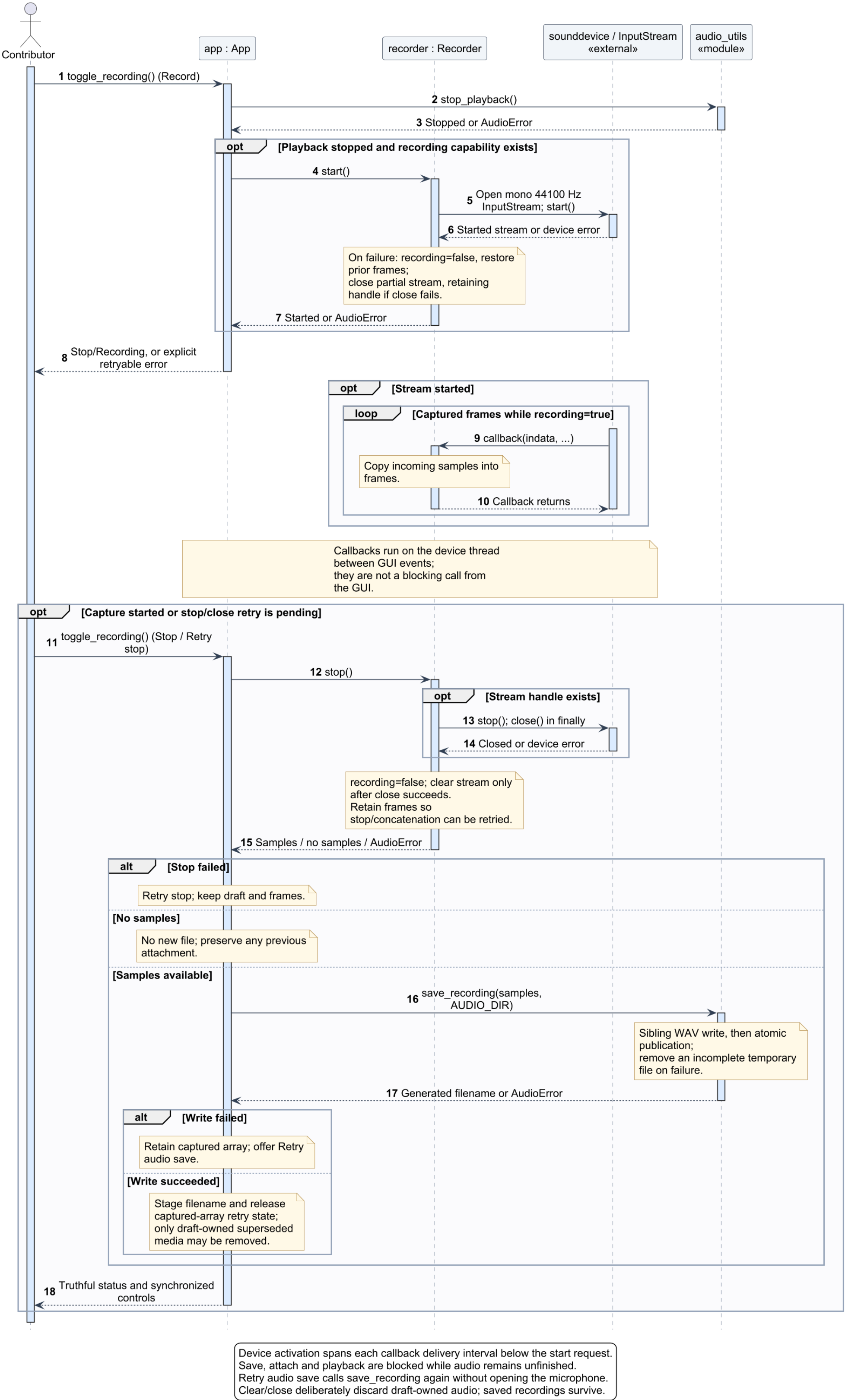
Desktop edits preserve ID, timestamp, contributor and audio reference. Text, glosses, topic, location, notes, language and lexical metadata are editable. Locks prevent interleaved writes, not semantic last-writer-wins conflicts.

Confirmed deletion without false success or media loss

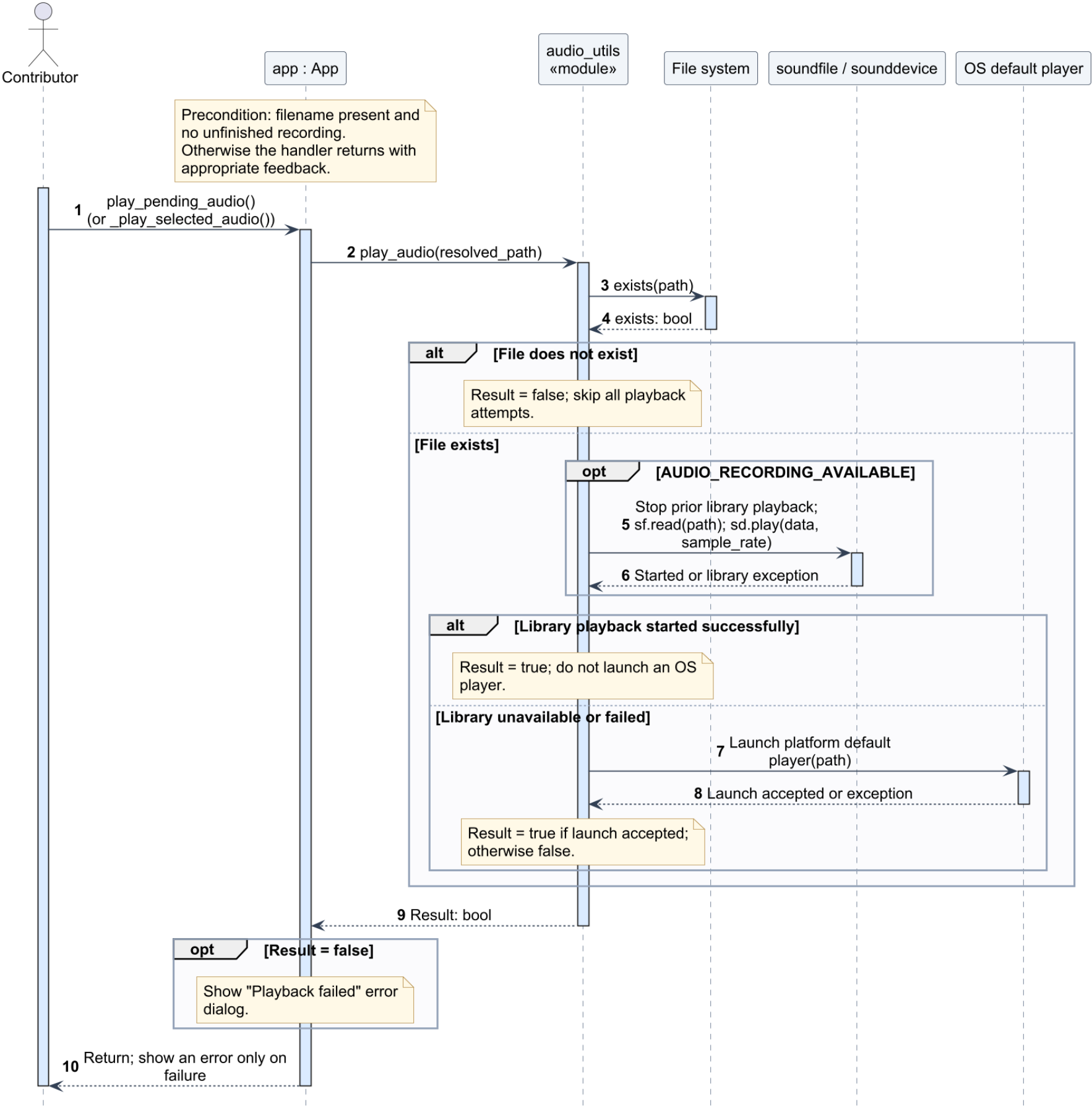


Confirmation is a decision, not an authorization or authentication check.
Atomic replacement protects write integrity; there is no undo stack.

Retryable microphone capture and WAV staging

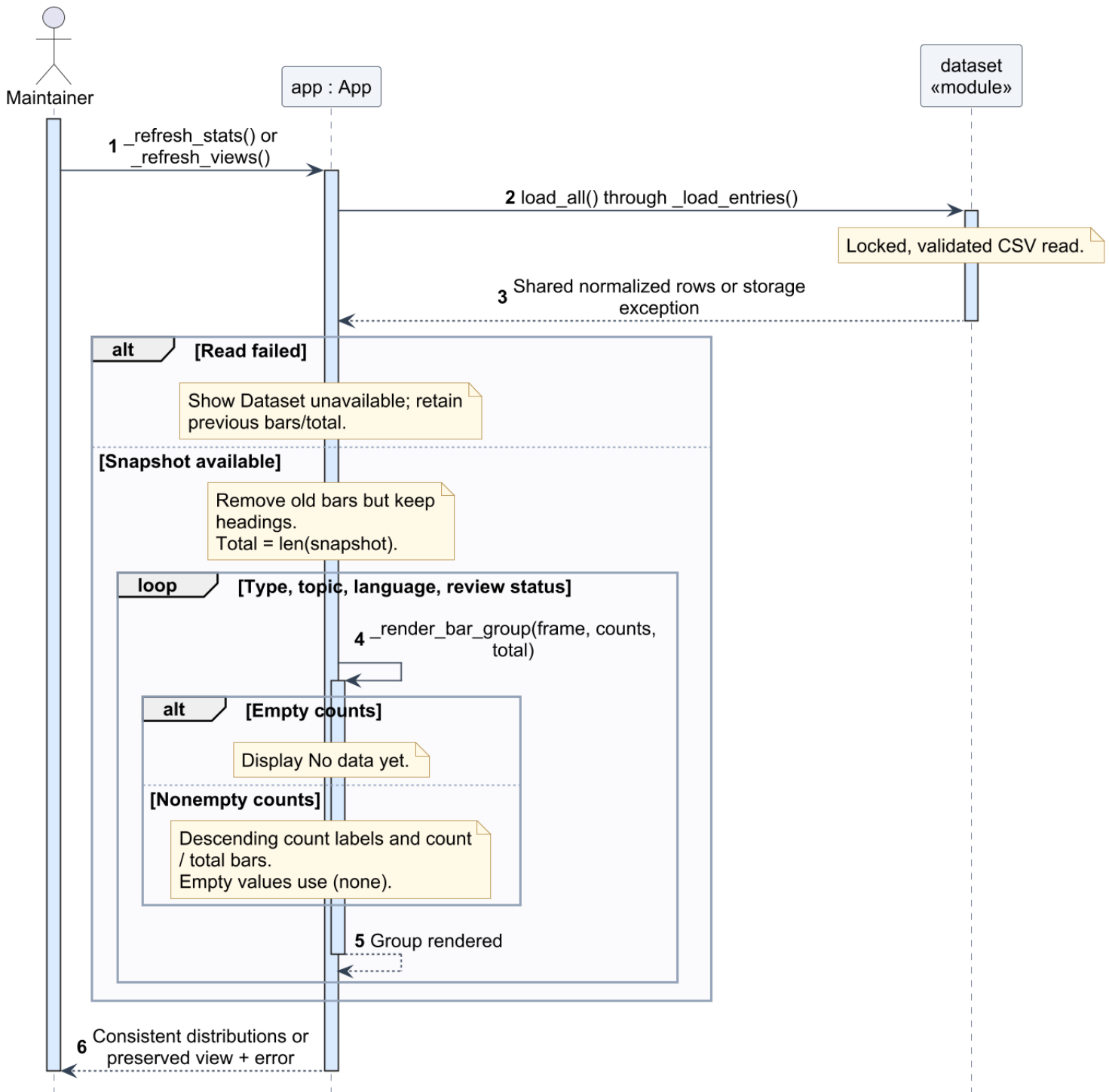


Play Audio with Library and OS Fallback



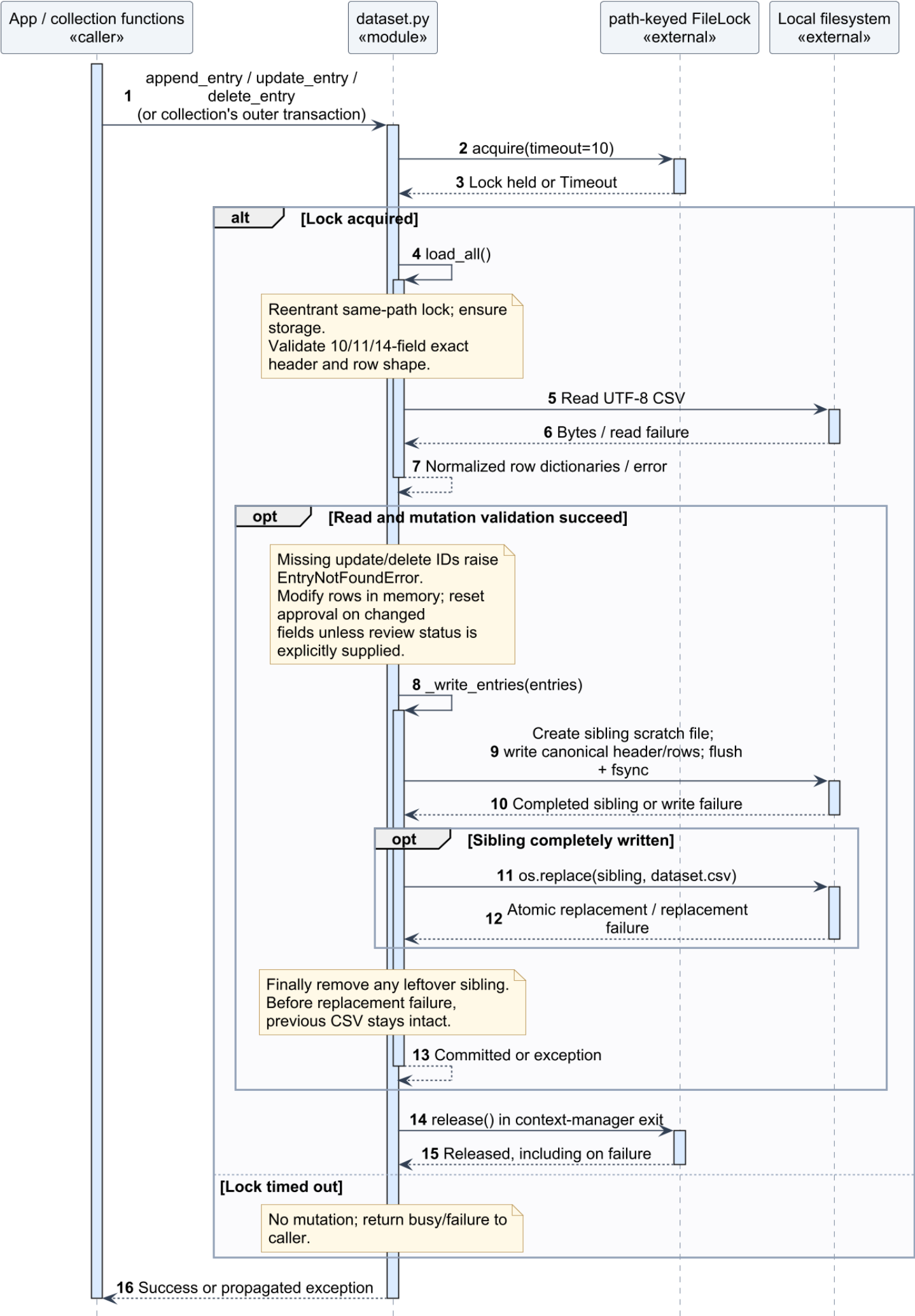
true means a playback request was accepted, not that sound was heard to completion. OS launch acceptance is not playback completion; media decoding is external. Expected audio errors are reported by the GUI; CSV is unchanged. Clear/close can stop in-process playback, not an external OS player's lifetime.

Consistent four-way statistics from one snapshot



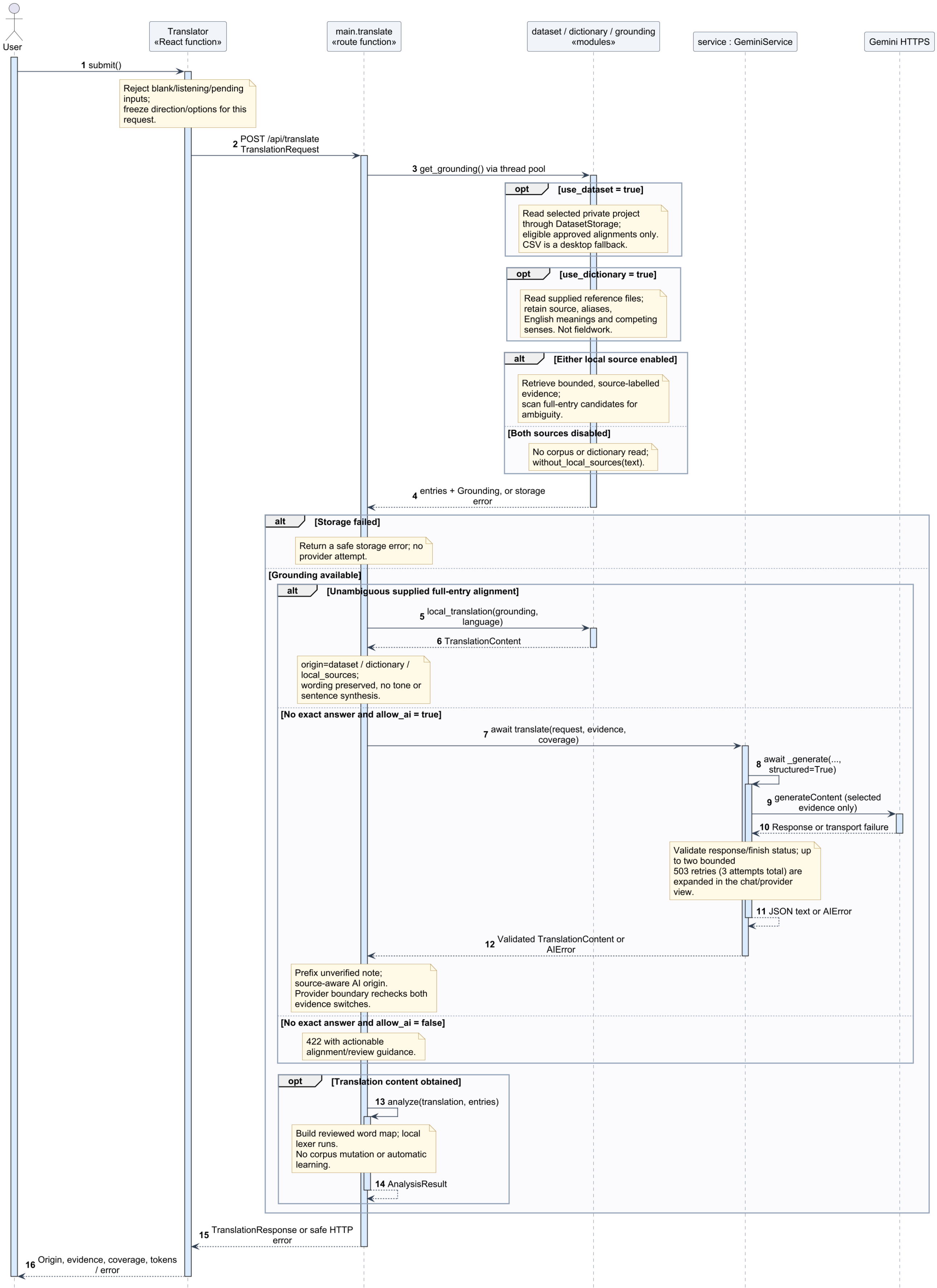
`_refresh_views` also supplies the same rows to count, recent and Browse views. Direct Stats refresh loads once; `count_by()` is not called by this handler. Performance evidence includes active-tab Tk idle rendering, not only CSV timing.

Shared locked CSV mutation and failure-safe replacement

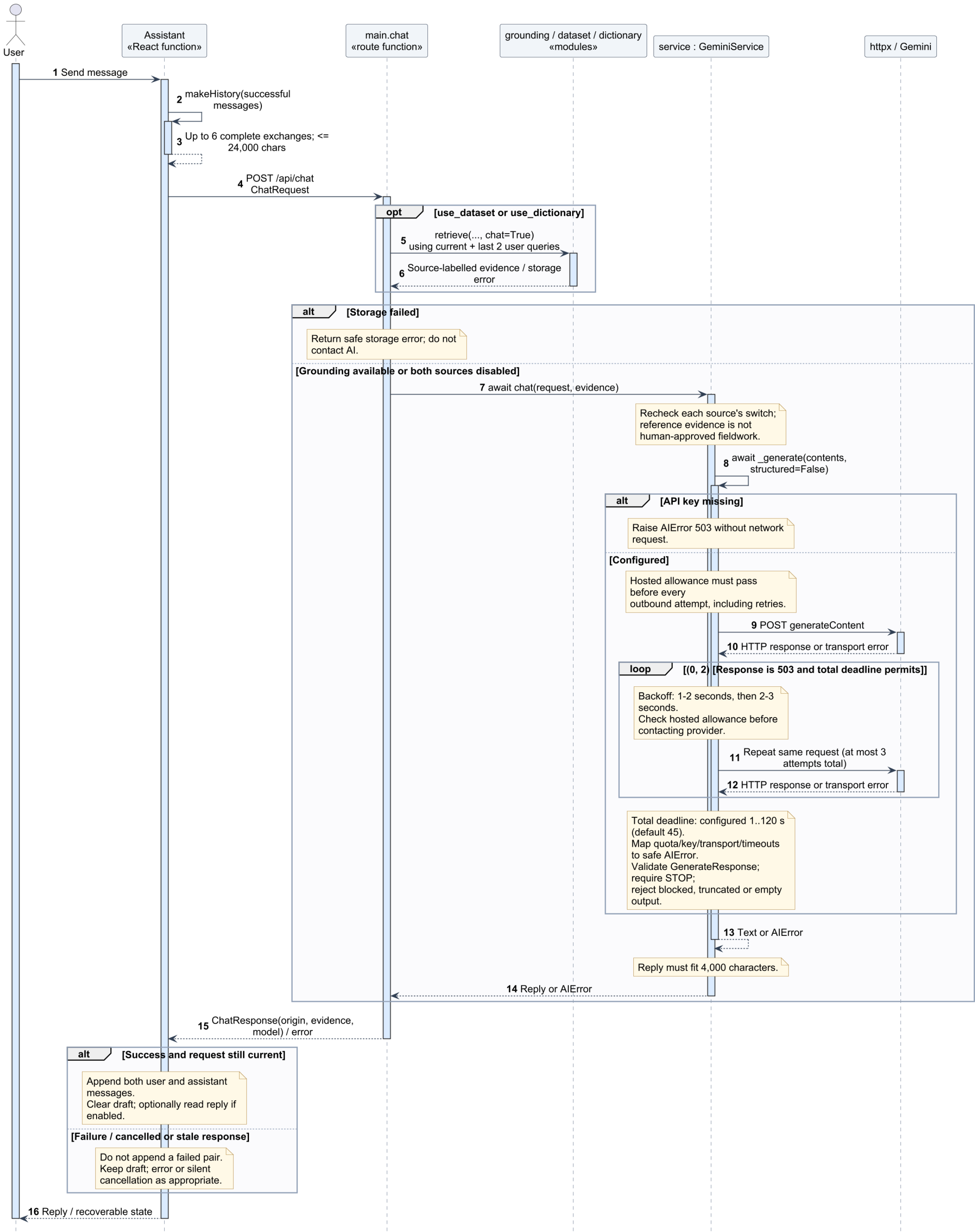


collection functions take the same outer lock around duplicate/ID checks and write. Lock activation shows the acquire/release call, not the duration of file ownership. Append also rewrites the full dataset; there is no unsafe direct append path. Protection covers cooperative users of this path, not external manual writers.

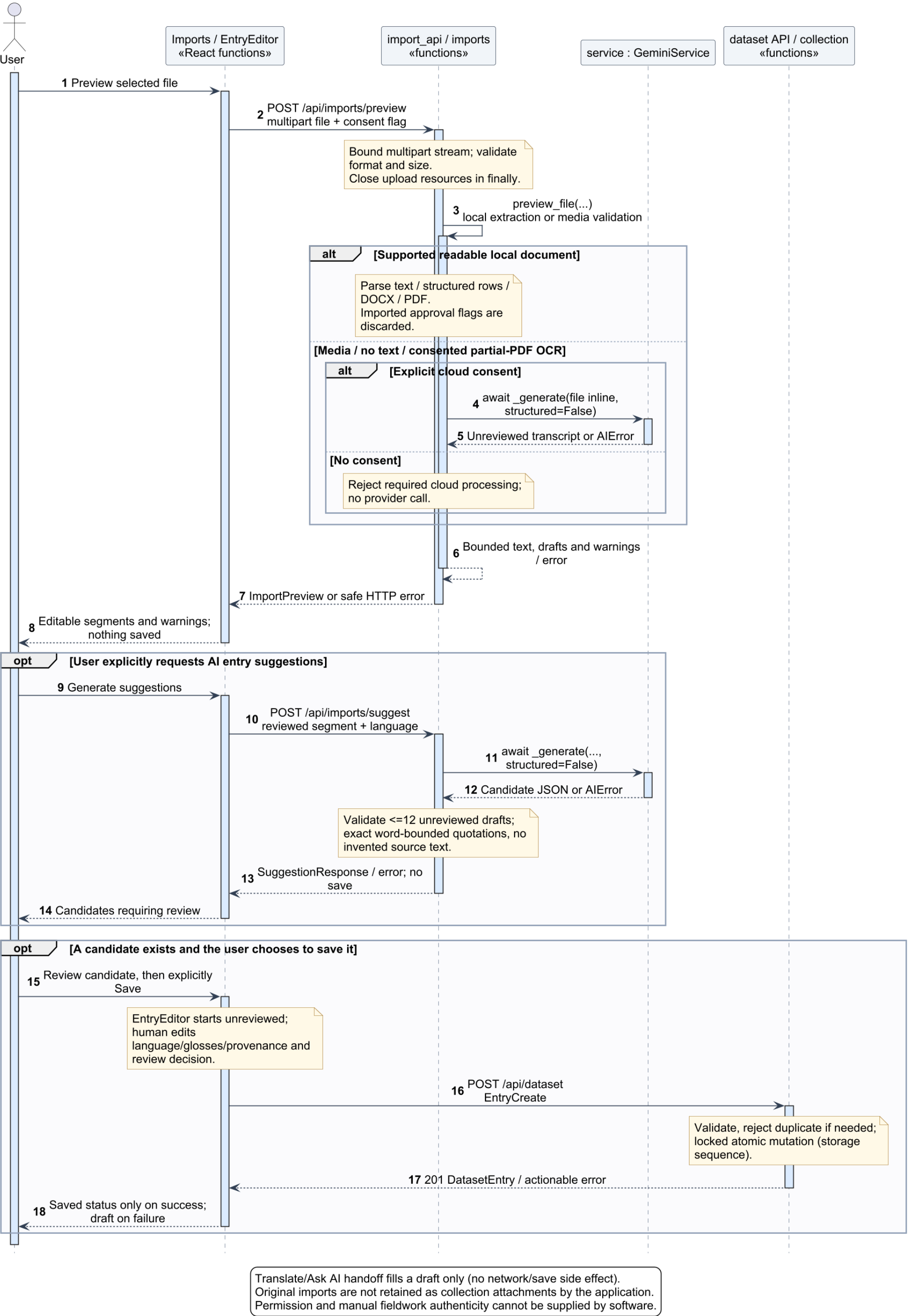
Web translation: enabled local sources, then optional AI



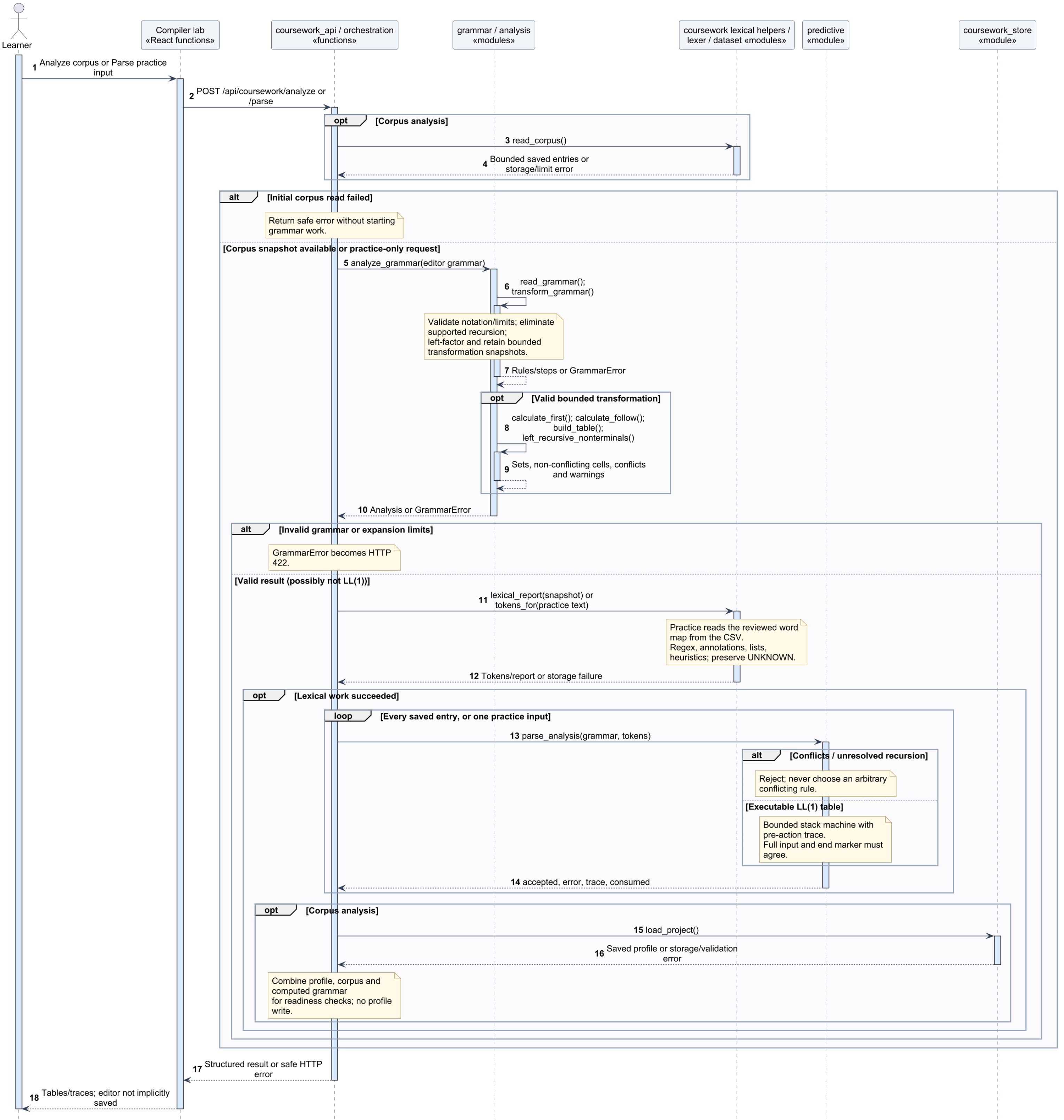
Explicit chat and bounded provider response handling



Import preview, explicit suggestions and human review/save

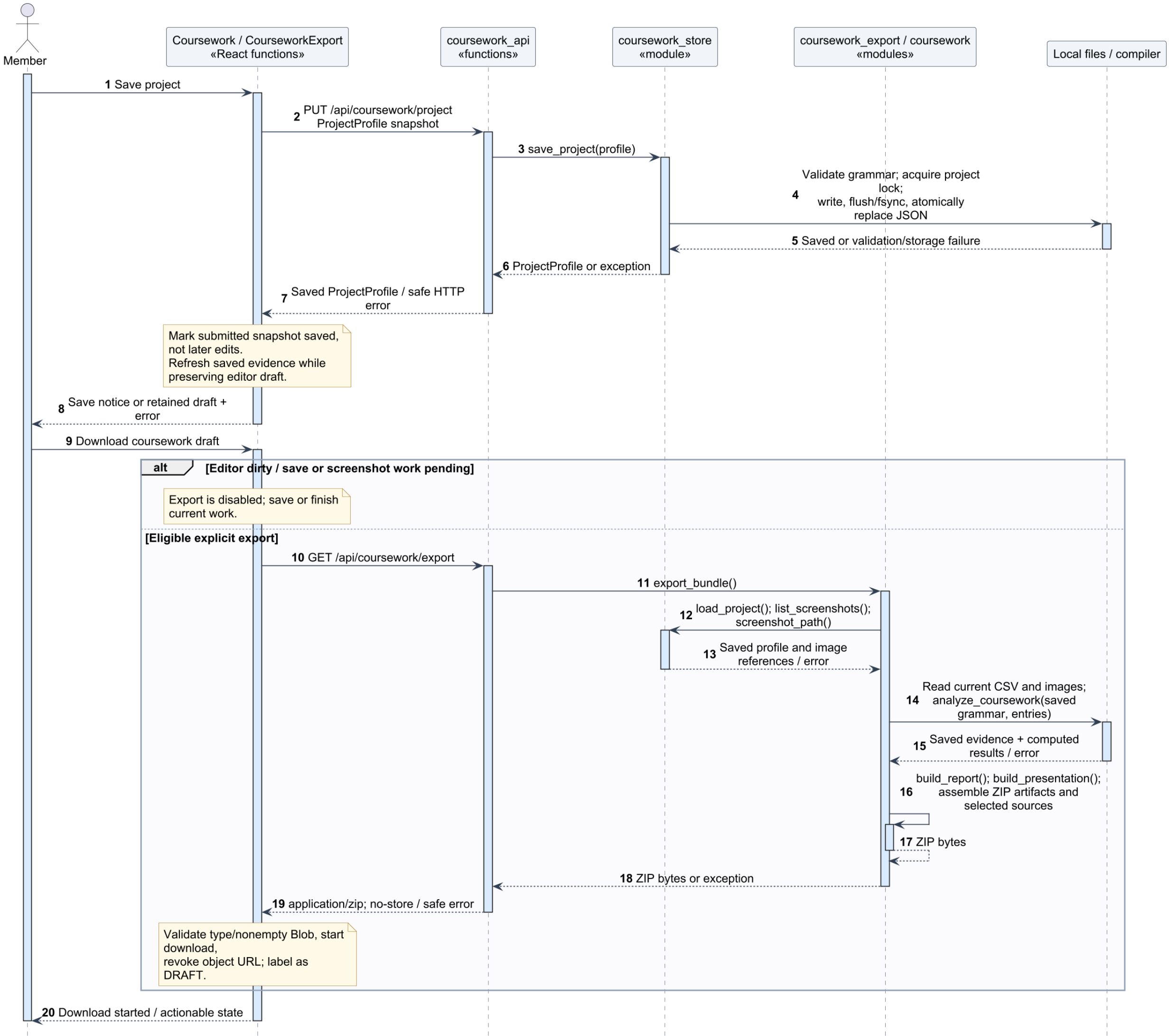


Deterministic grammar analysis and category-token parsing



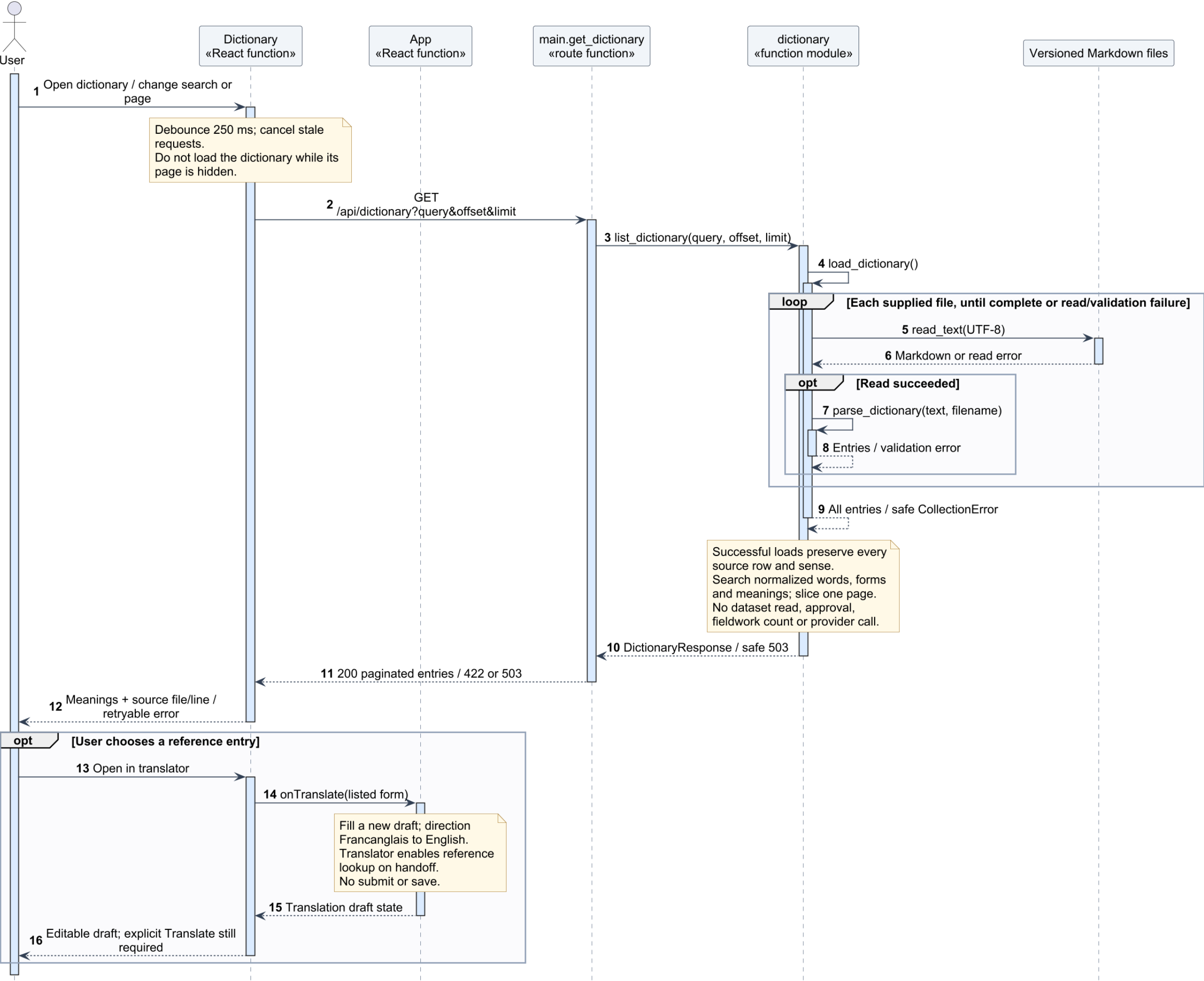
Module self-calls condense internal function boundaries.
No provider determines parsing. Corpus/grammar fit is not linguistic authenticity.
Existing corpus/media stay unchanged; absent storage can bootstrap an empty header.

Save coursework, then export only saved evidence

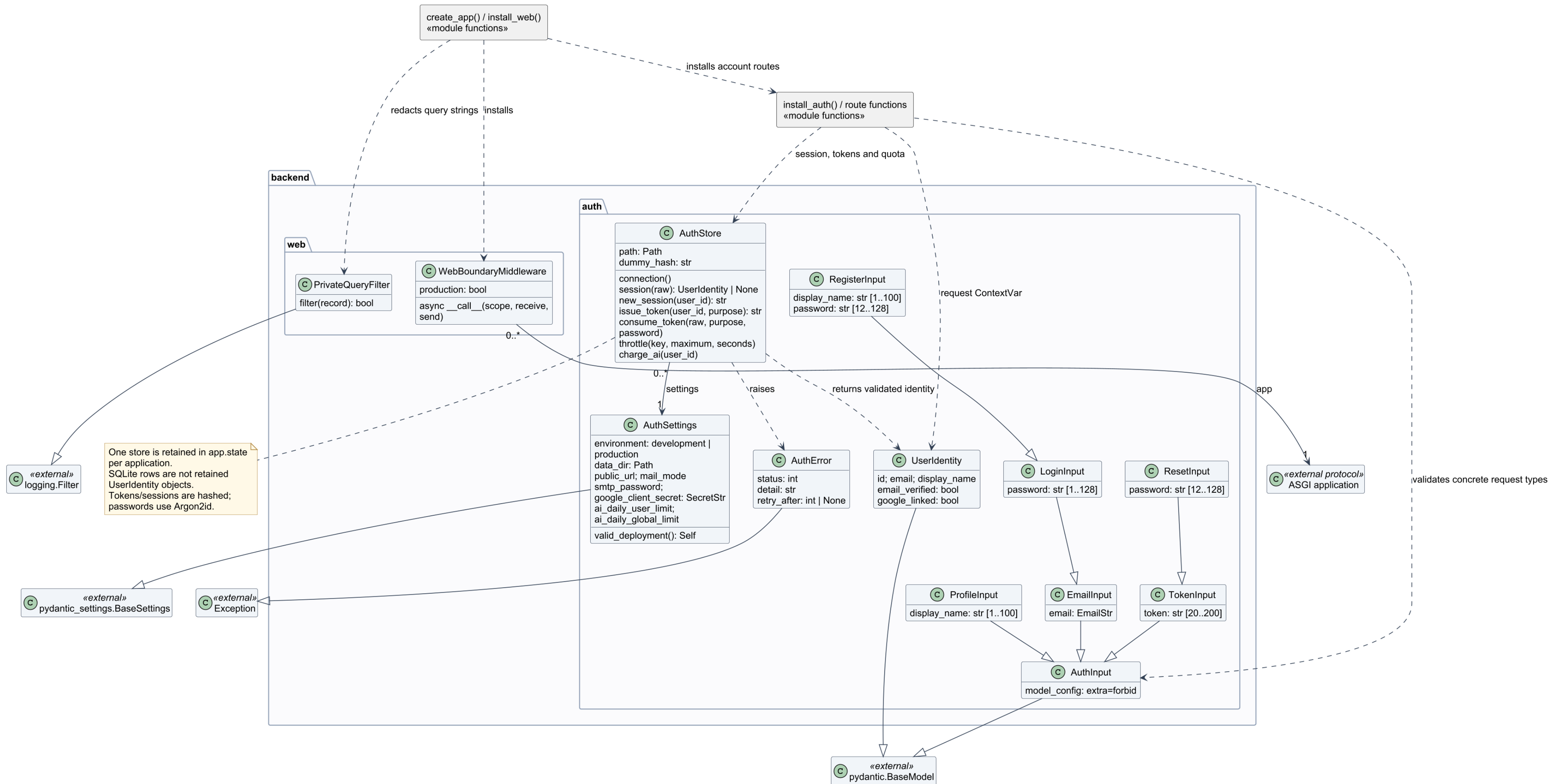


No unsaved practice input or AI answer is included.
ZIP includes actual corpus/provenance/profile: review before sharing.
Individual files are protected, but export is not a global multi-file transaction.
Missing research remains visible; 25 sections do not guarantee 25-30 final pages.

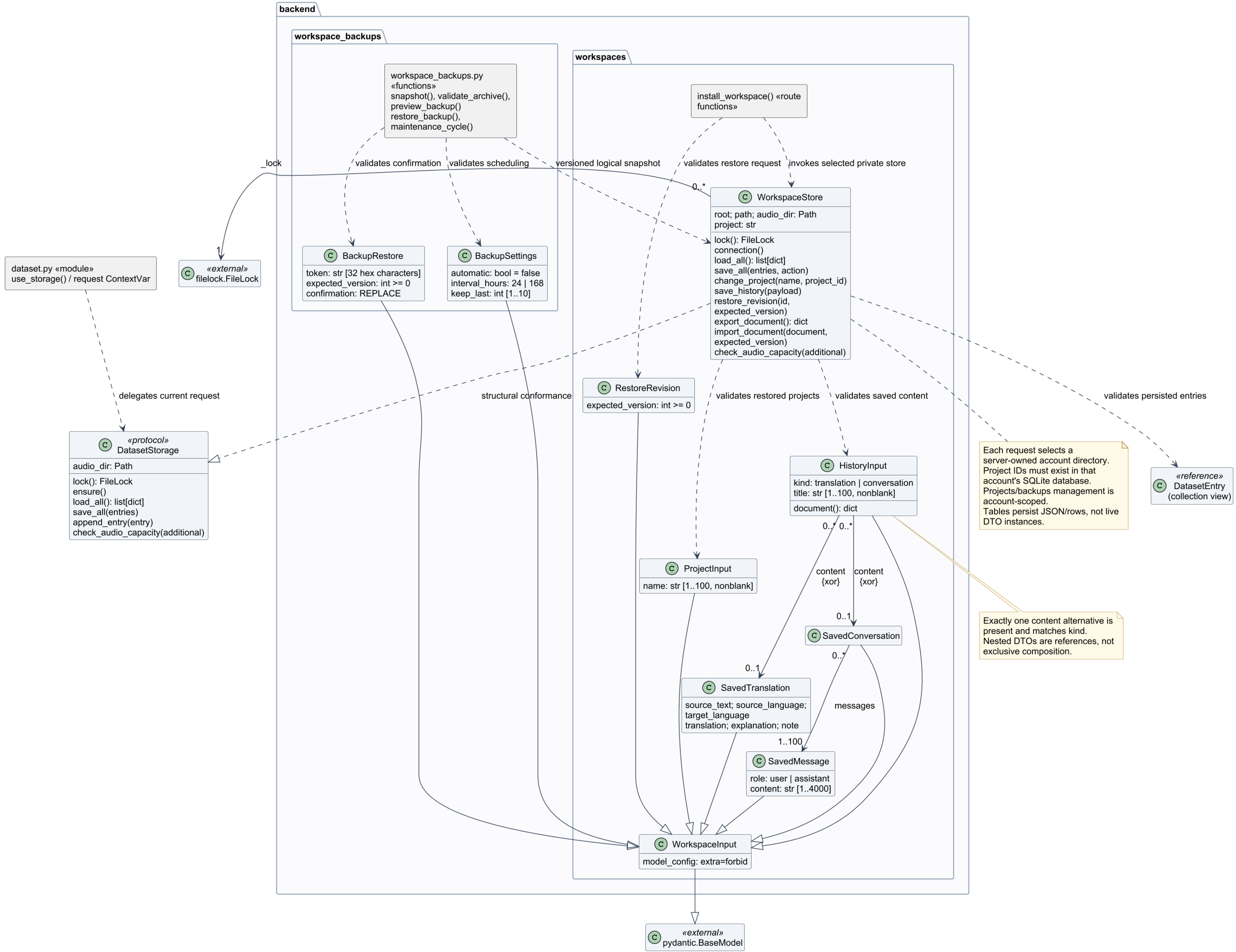
Reference lookup: separate sources, explicit draft handoff



Hollow triangles: inheritance. Dashed arrows: dependencies, not object ownership.
Associations describe actual stored references and include multiplicities.
Secrets stay server-side; UserIdentity is the public account DTO.

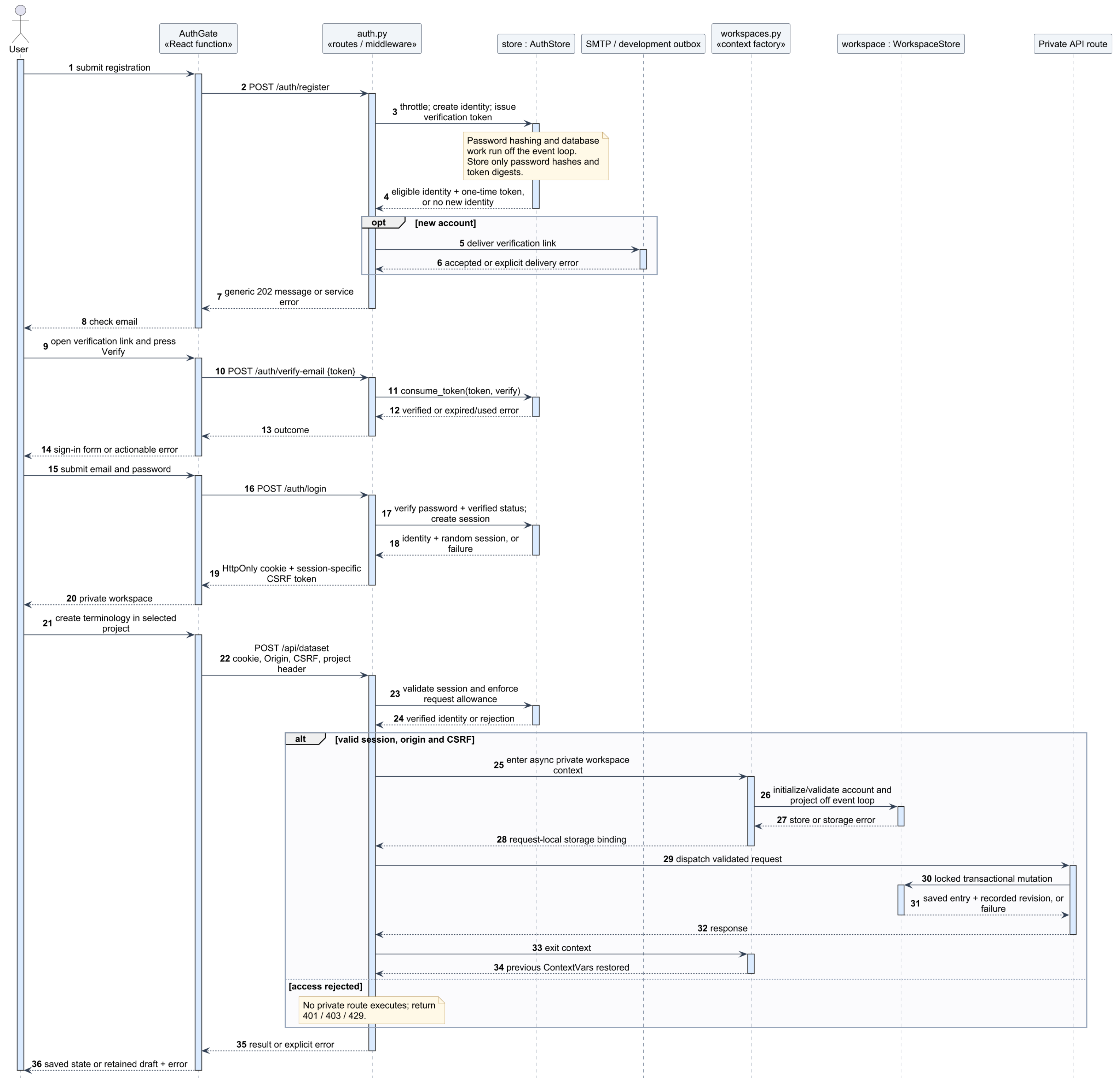


Private projects, saved work and recovery: actual classes

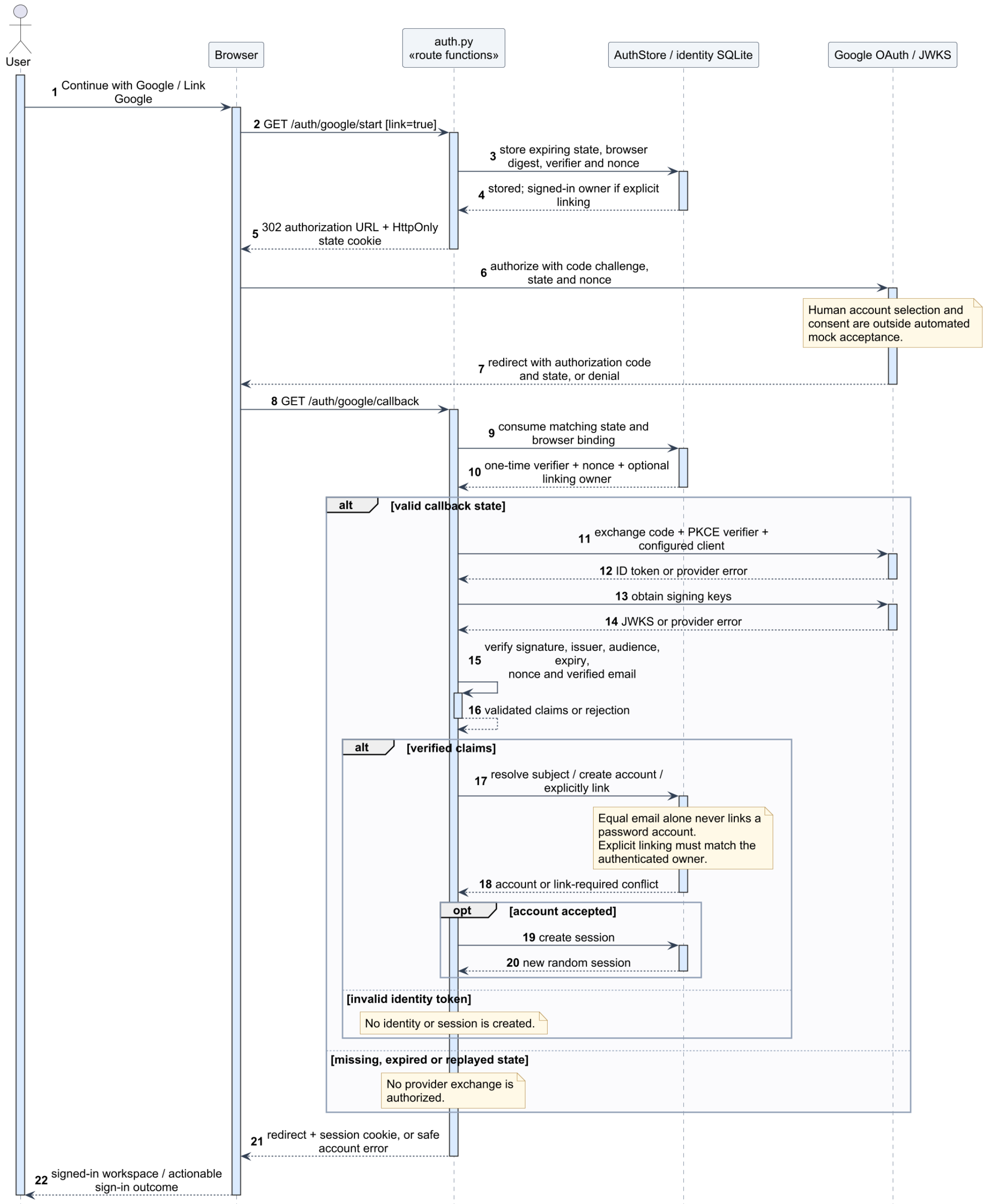


Generalization/realization point toward the base contract.
The CSV fallback is used only without a hosted storage context.
Audio and SQLite are coordinated, not one filesystem-wide ACID transaction.

Email account access and private request binding

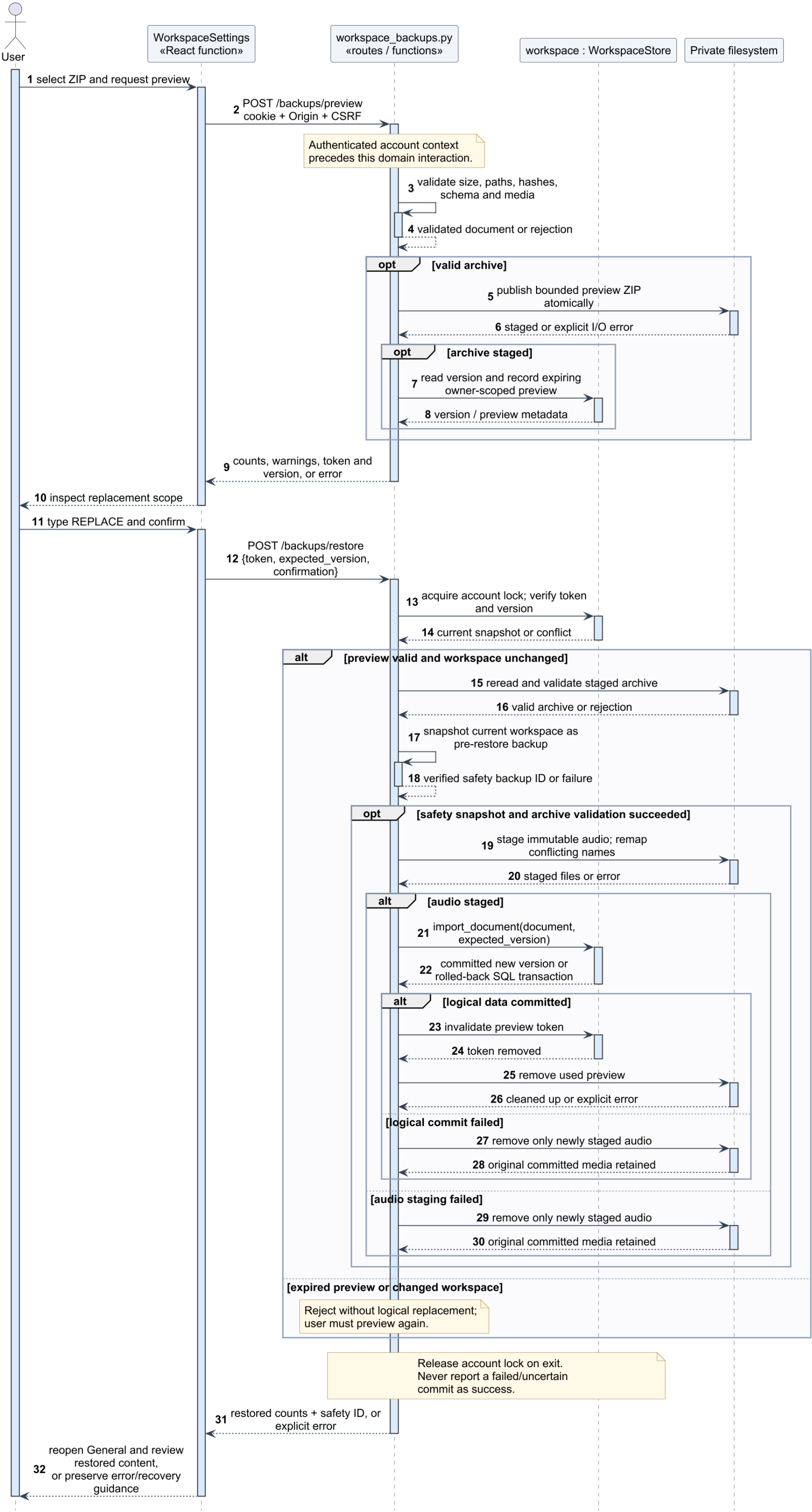


Google sign-in / explicit linking: authorization-code flow with PKCE



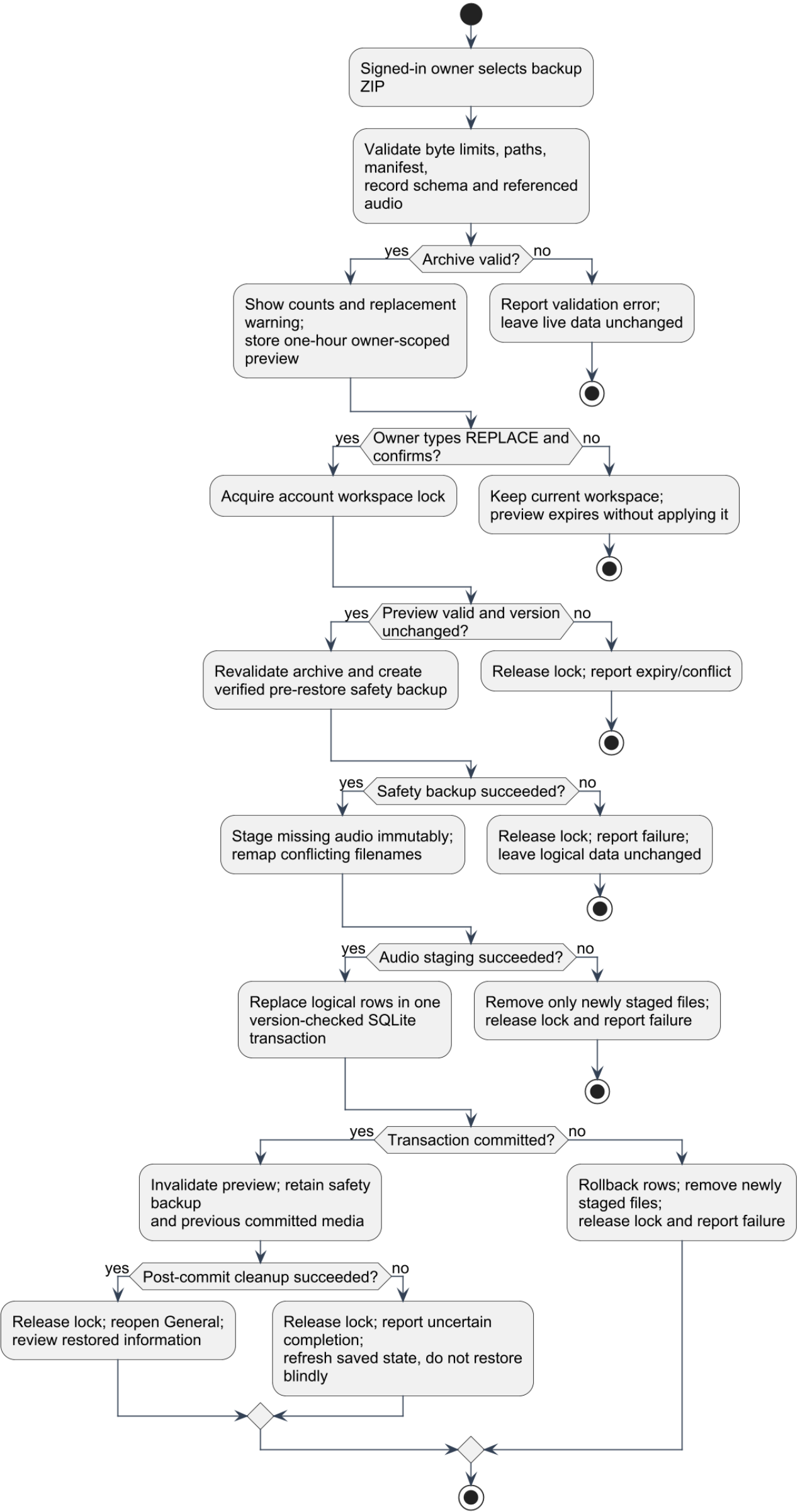
Redirects are HTTP replies; following them begins a new request. Code, verifier, nonce and credentials are not written to browser storage. Query values are redacted from application access logs.

Validated account backup restore with a safety copy



Backups replace logical workspace data, not account credentials or server settings.
Late client cancellation does not promise server rollback.
Existing committed media remains available for recovery; this is not a cross-file ACID transaction.

Private workspace restore: guarded replacement and failure paths



Branch guards are exclusive; each final node ends this attempt.
A cleanup error is explicit and does not imply a committed transaction rolled back.
This activity does not delete account credentials or other users' workspaces.